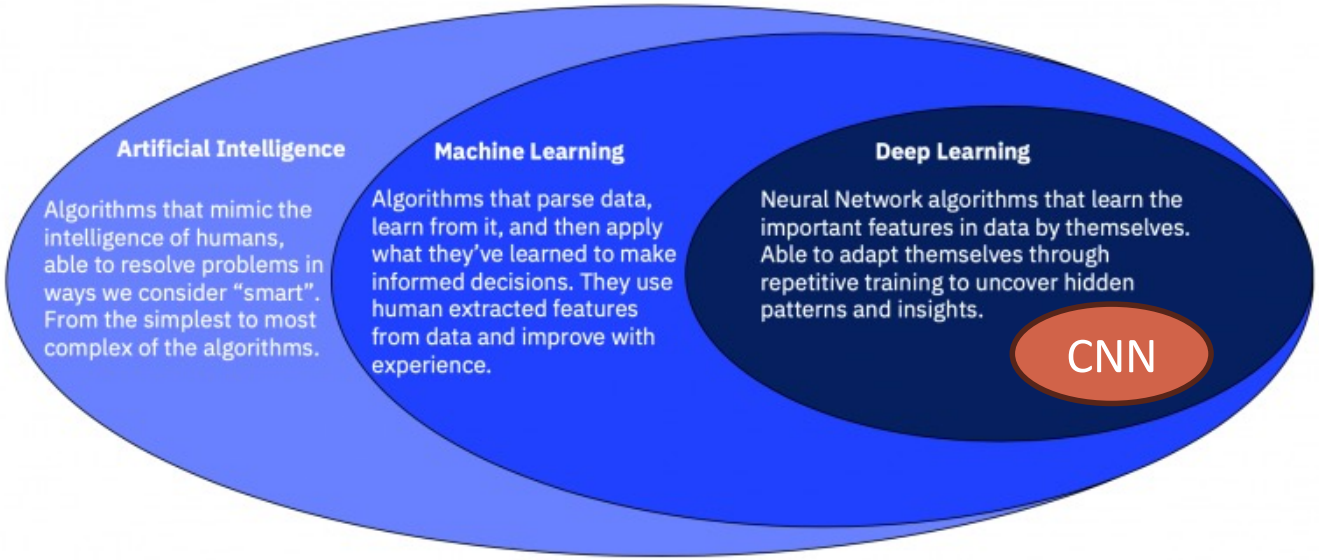
Convolutional Neural Networks

Prof Dr Melike Şah Direkoğlu

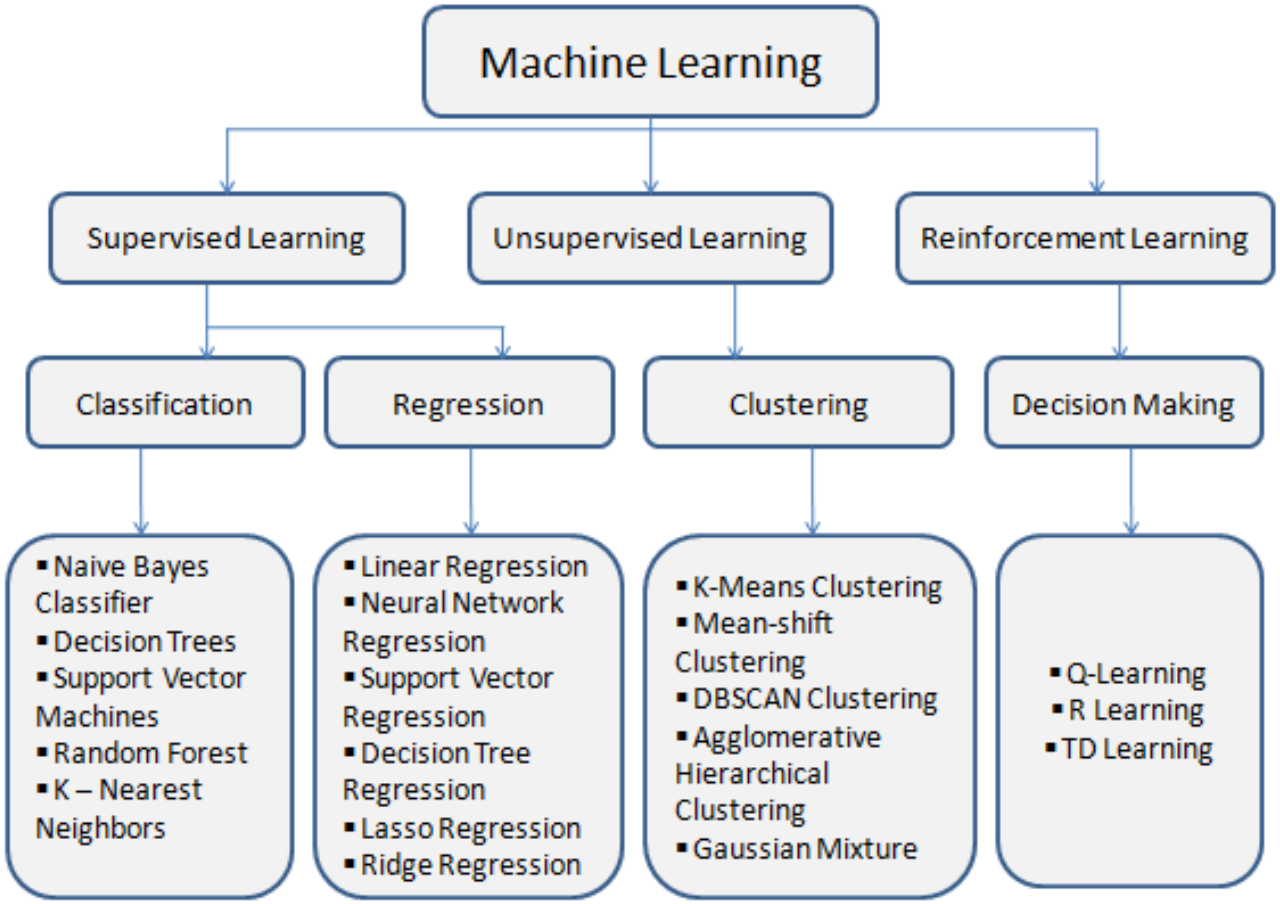
Slide credit: Cem Direkoğlu, Weifeng Li, Victor Benjamin, Xiao Liu, and Hsinchun Chen

AI, ML, DL, CNN

- Artificial Intelligence (AI) is a program that mimics human intelligence (the general term)
- Machine Learning (ML) is a subset of AI that uses algorithms and the performance improves with more data (uses hand-selected features)
- Deep Learning (DL) is a subset of ML, where multi-layered neural networks learn from vast amounts of data themselves (automatic feature learning)
- Convolutional Neural Networks (CNNs) are deep neural networks with several hidden layers (including the convolutional layers)



ML Algorithms

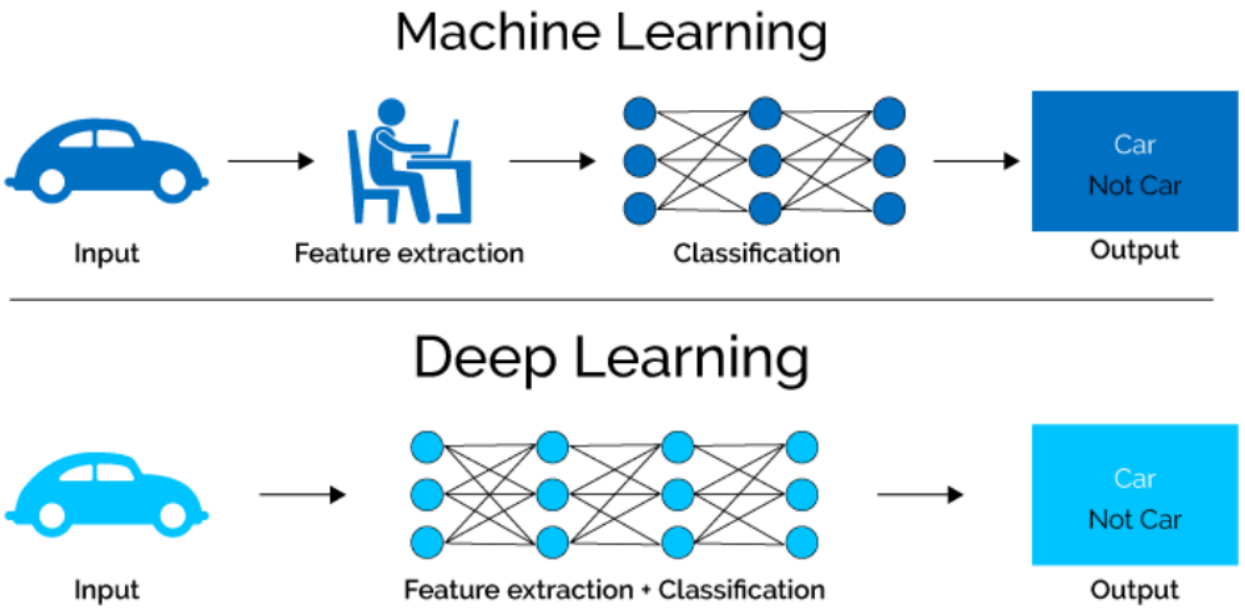


ML Algorithms (Cont.)

- **Supervised Learning:** In a supervised learning model, the algorithm learns on a **labeled dataset**
  - **In a supervised classification,** the algorithm use training data to categorize unseen data into different class labels (naïve bayes, SVM, random forest, etc.).
  - **In a supervised regression,** regression algorithms are used to predict continuous values such as price, age, stock market etc. (linear regression, support vector regressions, etc.)
- **Unsupervised Learning:** In an unsupervised learning model, the algorithm learns on an **unlabeled dataset** and tries to make sense by extracting features, co-occurrence, and underlying patterns on its own (K-means, mean shift, etc.)
- **Reinforcement Learning:** Model decides what actions to take in order to perform a given task that's why it is bound to learn from the experience itself.

ML vs DL

- In ML feature extraction is manual, whereas in DL algorithms can learn the features themselves.
- However, DL algorithms require vast amounts of data to train their models. Whereas in small domains, engineered features of ML can achieve good results as well.



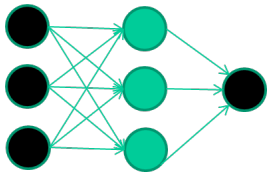
Deep Learning

What exactly is deep learning ?

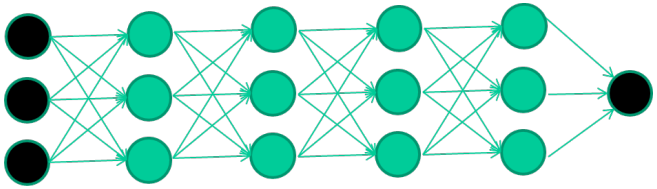
The short answers

1. **'Deep Learning' means** using a **neural network** with **several hidden layers** between input and output

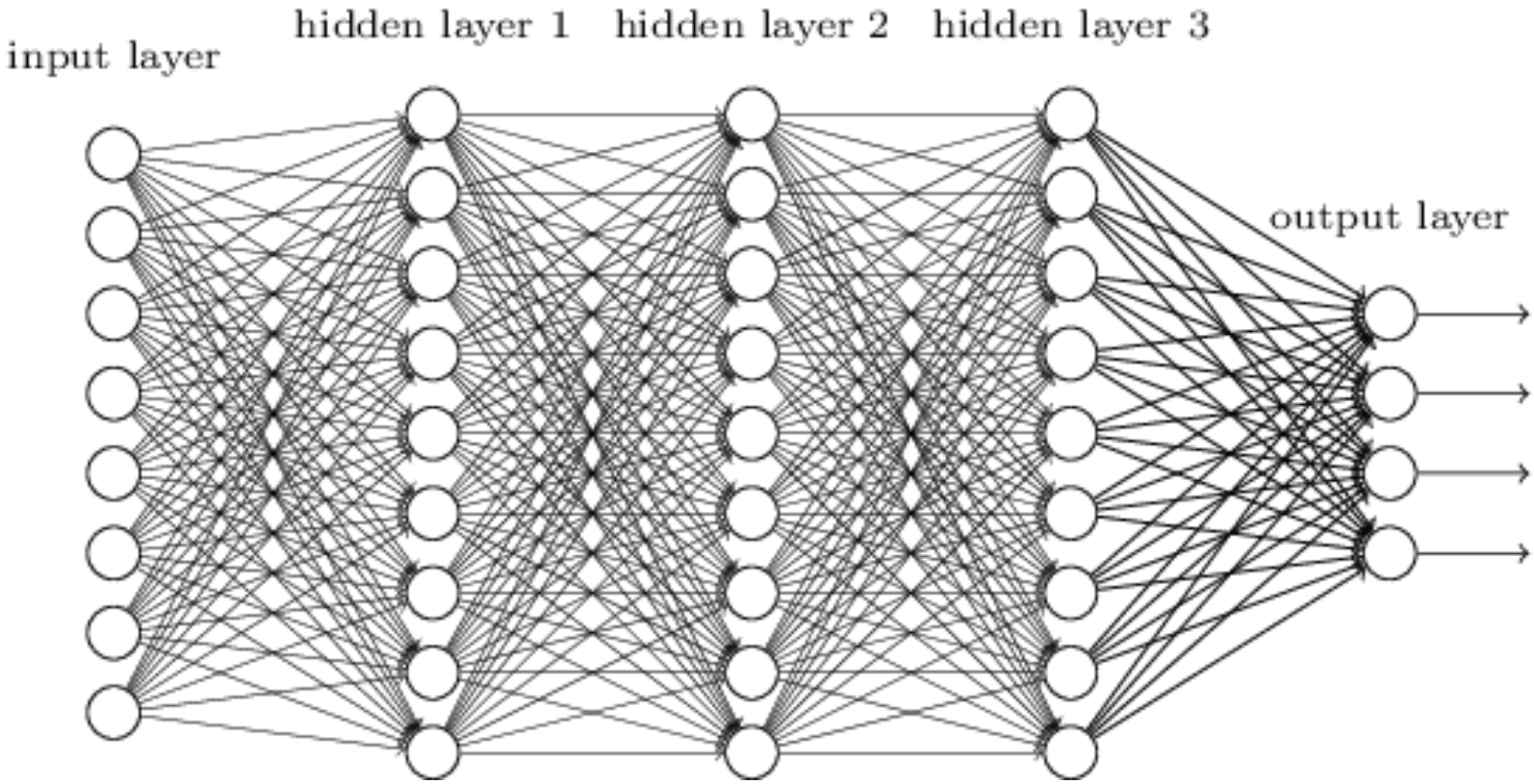
2. **The series of layers between input & output do feature identification and processing in a series of stages, just as our brains seem to.**



Extension to



Deep Neural Networks (Multilayer NNs)

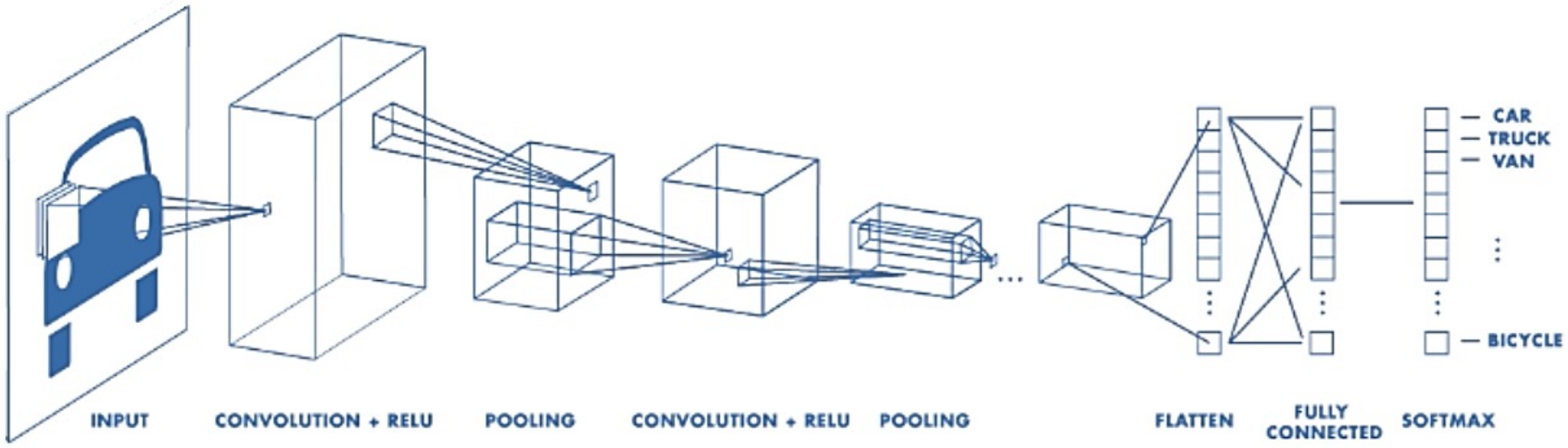


Deep means many hidden layers

Convolutional Neural Network (CNN)

What is CNN?

- In deep learning, a **convolutional neural network** (**CNN**, or **ConvNet**) is a class of deep neural networks, most commonly applied to analyzing visual imagery.
- A CNN consist of multiple layers that transform the input data into an output class/prediction. It also has a special layer called convolutional layer where its name come from.

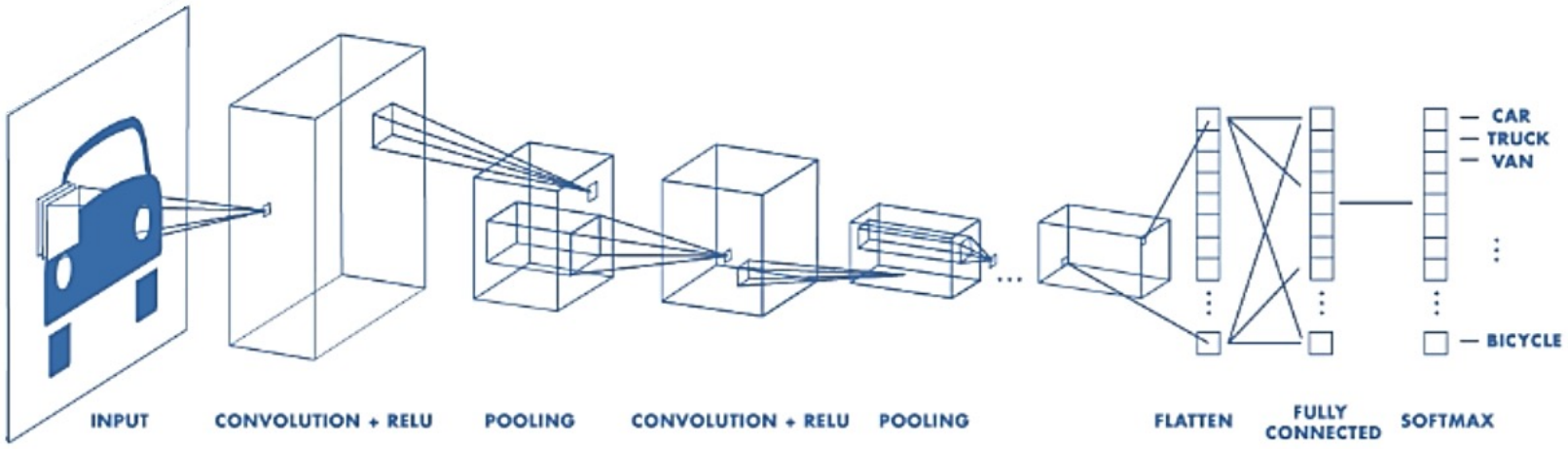


Building-blocks (layers) for CNN's

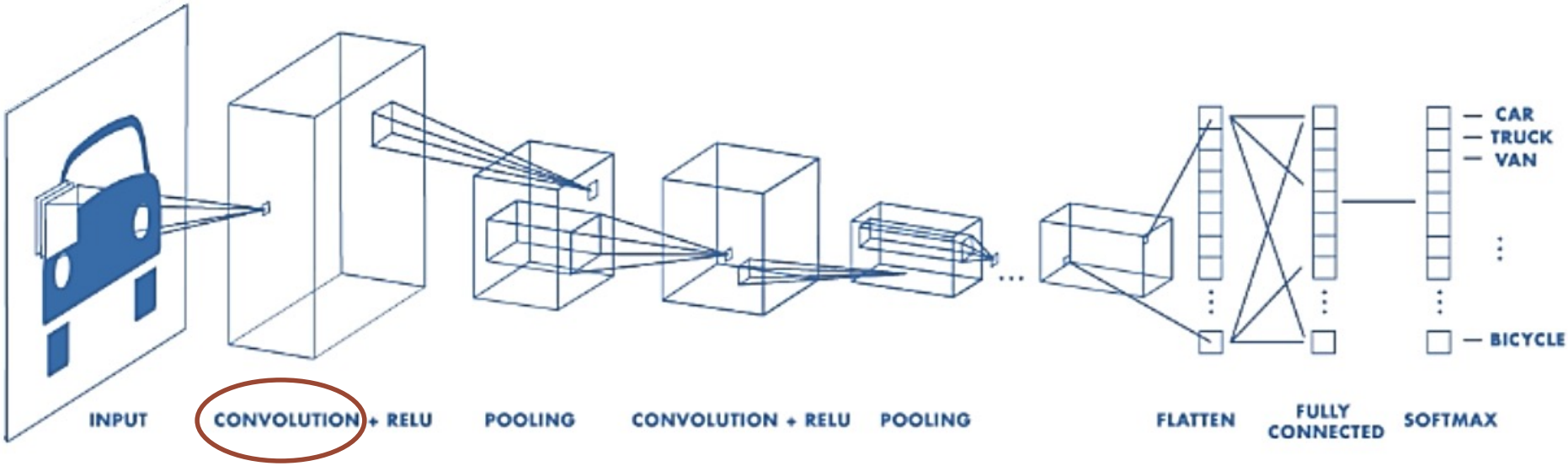
There are a few distinct types of layers:

- Convolutional layer
- Non-linear layer (Linear Rectifier Unit:RELU)
- Pooling layer (Max-Pooling, Min-Pooling, Avg-Pooling)
- Flattend and Fully connected layer
- Softmax layer

There are other layers such as batch_normalization layer, dropout layer, etc. as well.



Typical CNN Architecture

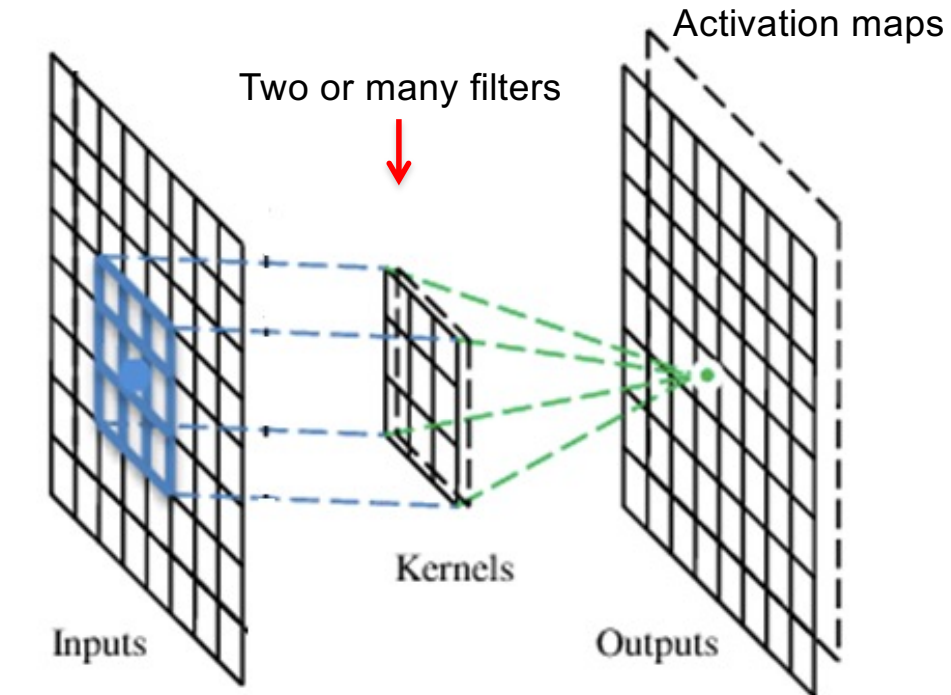


A Convolutional Layer

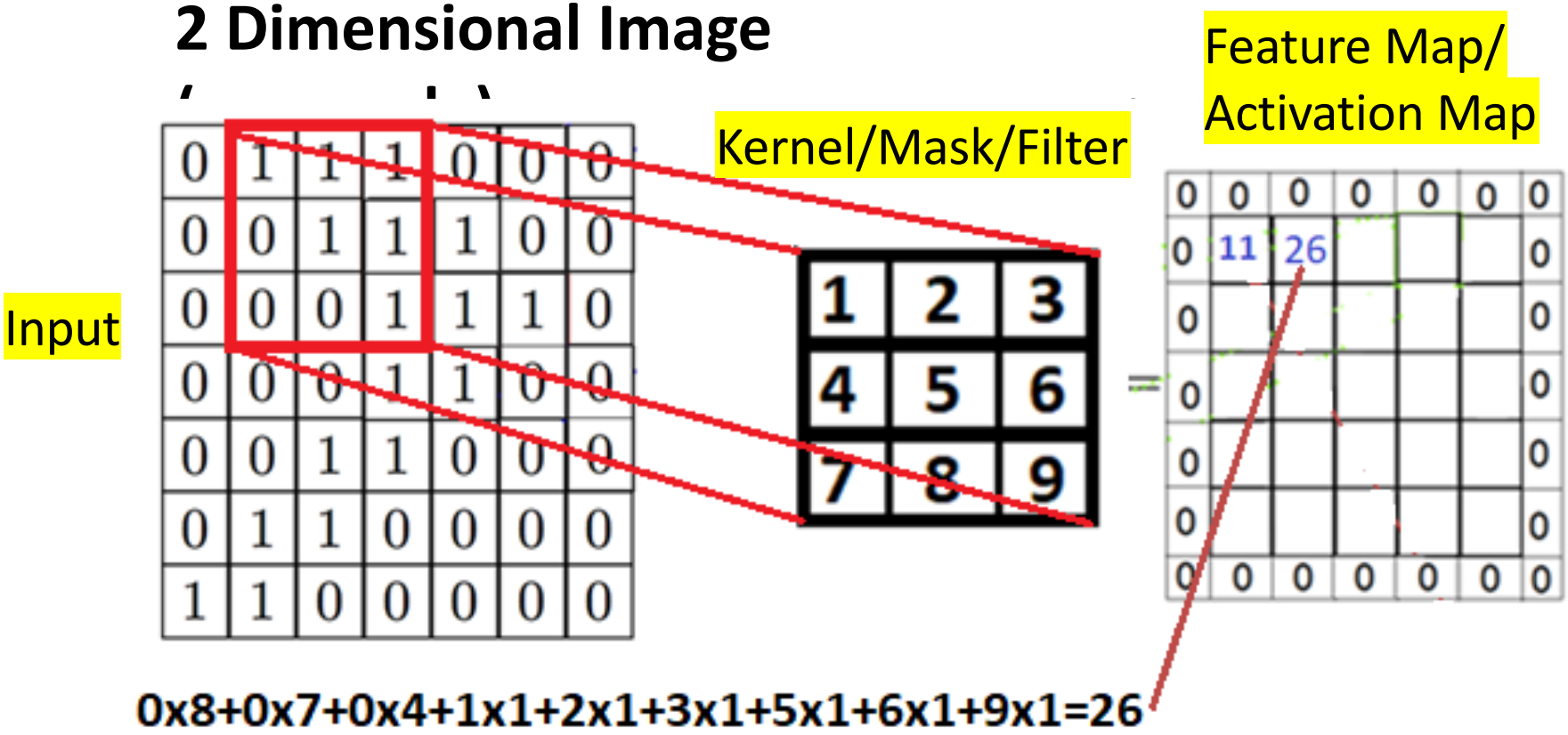
- A CNN is a neural network with some convolutional layers (and some other layers). **A convolutional layer has a number of filters that does the convolutional operation**.

- Filters are also called channels or kernels

* The network **will learn filters** that activate when they see some specific type of feature at some spatial position in the input. **Each filter is also called channel**.



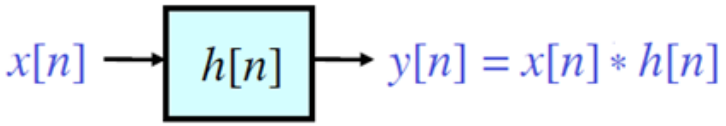
Convolution Operation



Linear filtering: Convolution

1- Dimensional Convolution

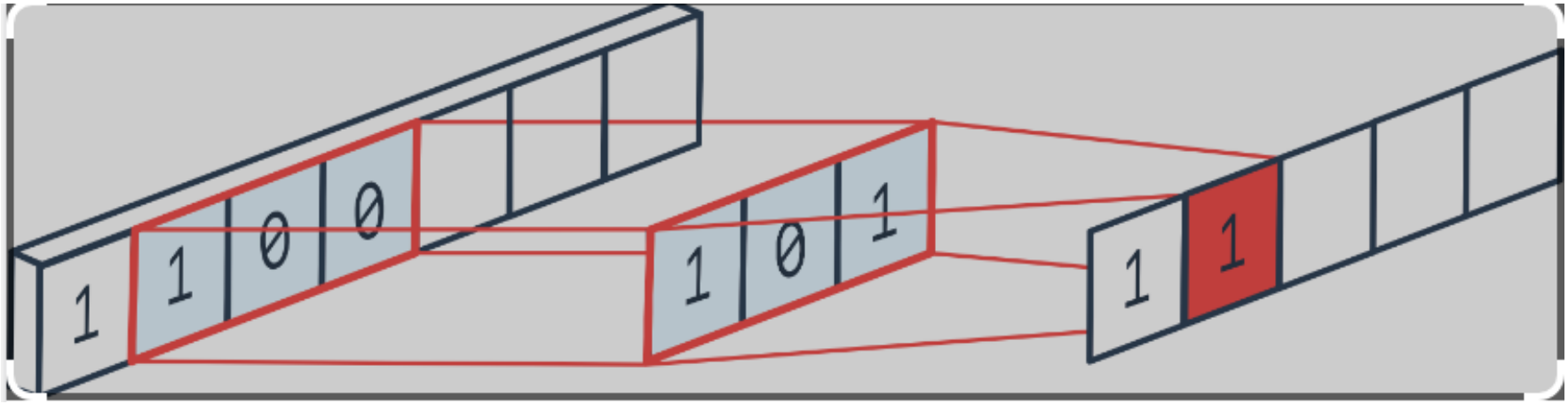
- **The Linear - Time Invariant (LTI) System is described by h[n]**



- $$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k] = x[n] * h[n]$$

- Properties
  - Commutative: $x[n] * h[n] = h[n] * x[n]$
  - Associative:
    $$(x[n] * h[n]) * y[n] = x[n] * (h[n] * y[n])$$
  - Distributive:
    $$h[n] * (x[n] + y[n]) = h[n] * x[n] + h[n] * y[n]$$

Convolution 1D Vector



1x1 + 1x0 + 0x1 = 1

1x1 + 0x0 + 0x1 = 1

Convolution in 2-Dimensions

$$y[n,m] = x[n,m] * h[n,m] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} x[k,l]h[n-k,m-l]$$

h[n,m] = “kernel” or “mask” or “filter”

x[n,m] is the input image

y[n,m] is the output image (Feature map in CNN)

Example

x[n,m]

m →

n ↓

0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	1	1	0	0	0	0
1	1	0	0	0	0	0

h[n,m]

m →

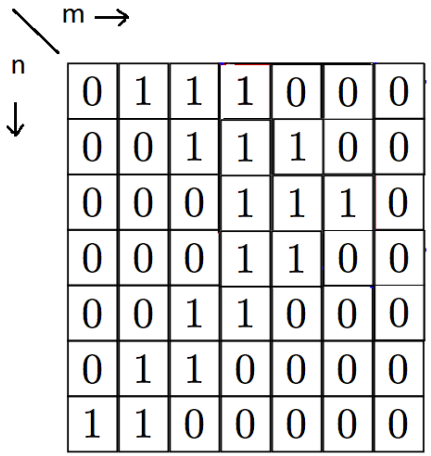
n ↓

1	2	3
4	5	6
7	8	9

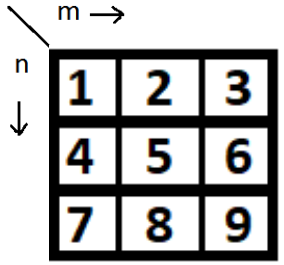
Find y[n,m] = x[n,m]*h[n,m].

Example: Linear Filtering with Convolution

x[n,m]



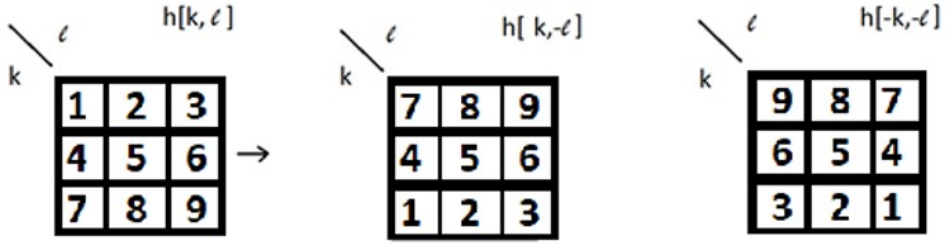
h[n,m]



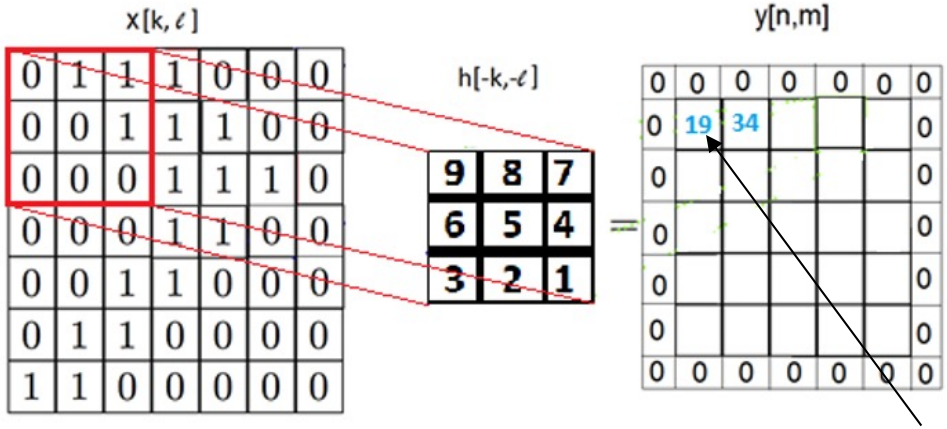
Find y[n,m] = x[n,m]*h[n,m].

$$y[n,m] = x[n,m] * h[n,m] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} x[k,l]h[n-k,m-l]$$

Solution



Flip h[k,l] vertically and horizontally



0x9 + 8x1 + 7x1 + 0x6 + 0x5 + 1x4 + 0x3 + 0x2 + 0x1 = 19

Cross-correlation

- In Convolutional Neural Networks (CNNs), the operation that is often referred to as "convolution" is technically a mathematical operation known as cross-correlation.
- The term "convolution" is used in the context of signal processing and mathematics, where the filter is flipped (rotated 180 degrees) before applying it to the input signal.
- However, in most deep learning frameworks and CNN implementations, the operation is actually cross-correlation, where the filter is not flipped.

Cross-correlation

In signal processing, cross-correlation is a measure of similarity of two signals.

image x[n,m]

Kernel/mask h[n,m]

Example: Linear Filtering with Cross-correlation

x[n,m]

m →

n ↓

0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	1	1	0	0	0	0
1	1	0	0	0	0	0

h[n,m]

m →

n ↓

1	2	3
4	5	6
7	8	9

Find $y[n,m] = x[n,m] \otimes h[n,m]$.

$$y[n,m] = x[n,m] \otimes h[n,m] = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} x[k,l]h[n+k,m+l]$$

Solution

Filter is not flipped!

x[k, ℓ]

0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	1	1	0	0	0	0
1	1	0	0	0	0	0

h[k, ℓ]

1	2	3
4	5	6
7	8	9

y[n,m]

0	0	0	0	0	0	0
0	11	26				0
0						0
0						0
0						0
0						0
0	0	0	0	0	0	0

0x9 + 0x8 + 0x7 + 0x4 + 0x1 + 0x5 +2x1 +3x1 + 6x1 = 11

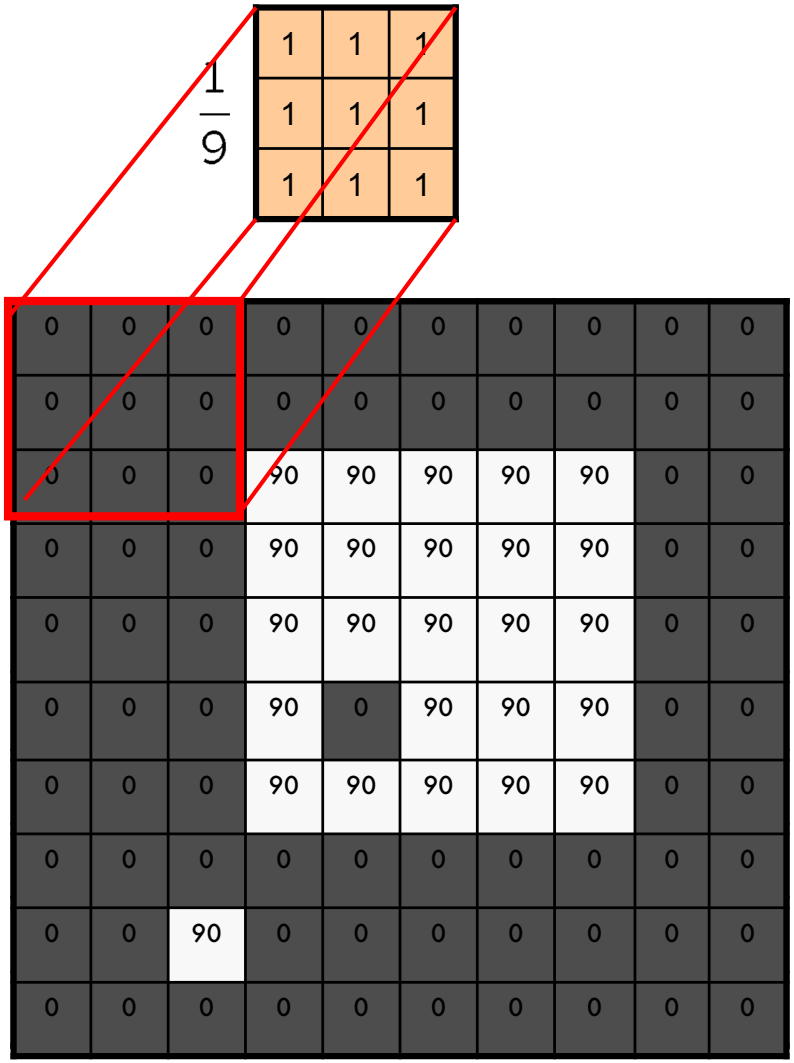
Convolution Operation in CNN

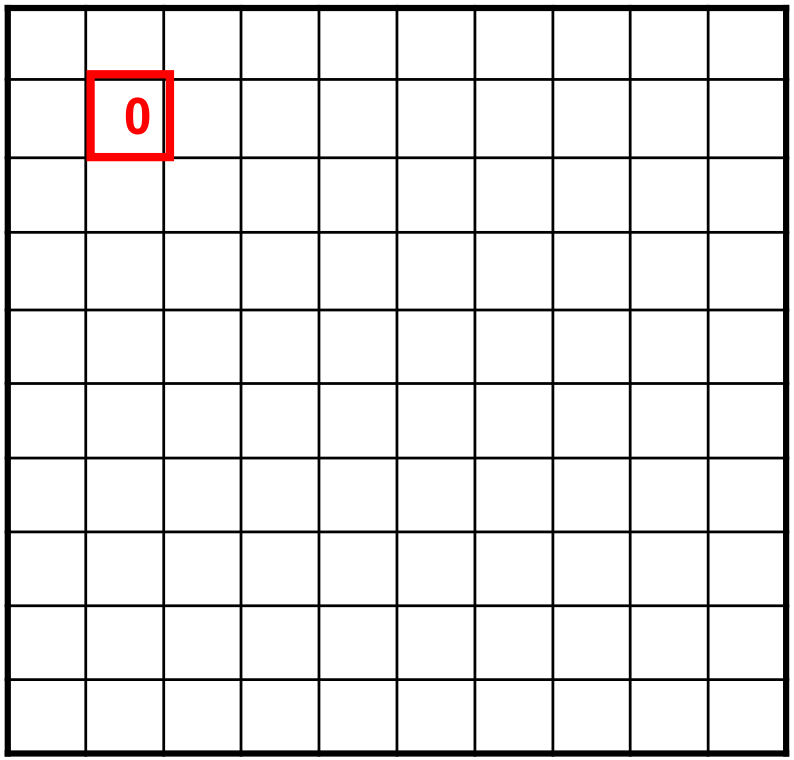
- In most deep learning frameworks and CNN implementations, the operation is actually cross-correlation, where the filter is not flipped.

Example: box filter

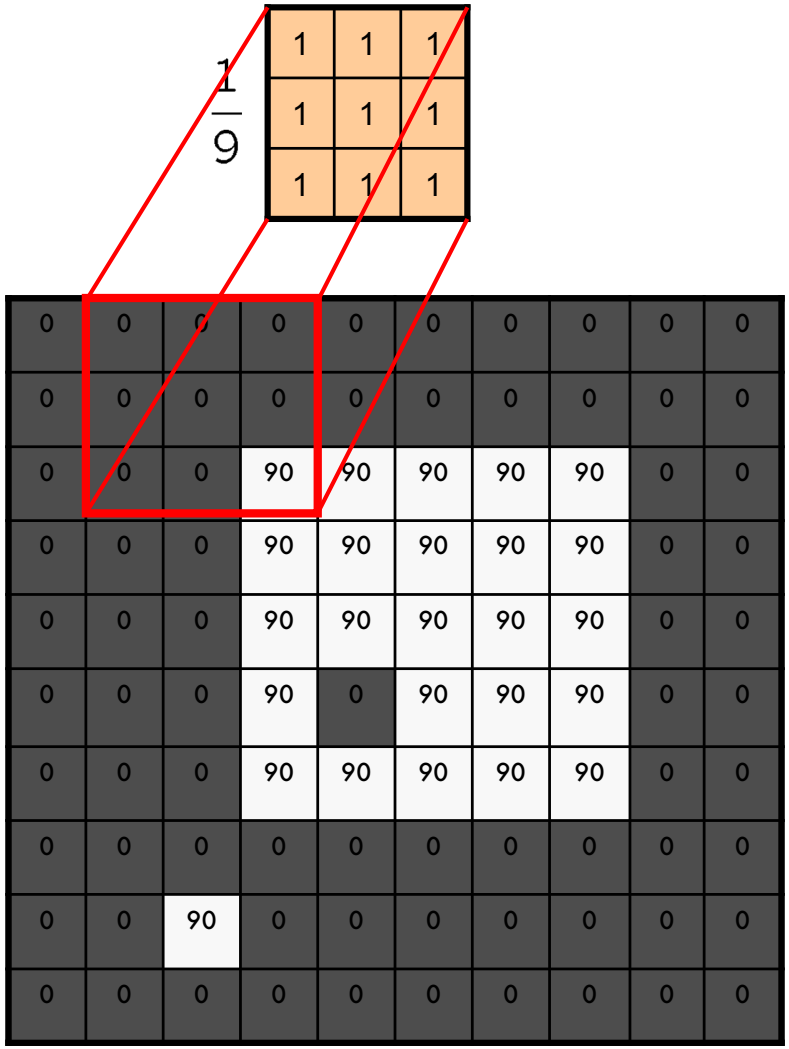
$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

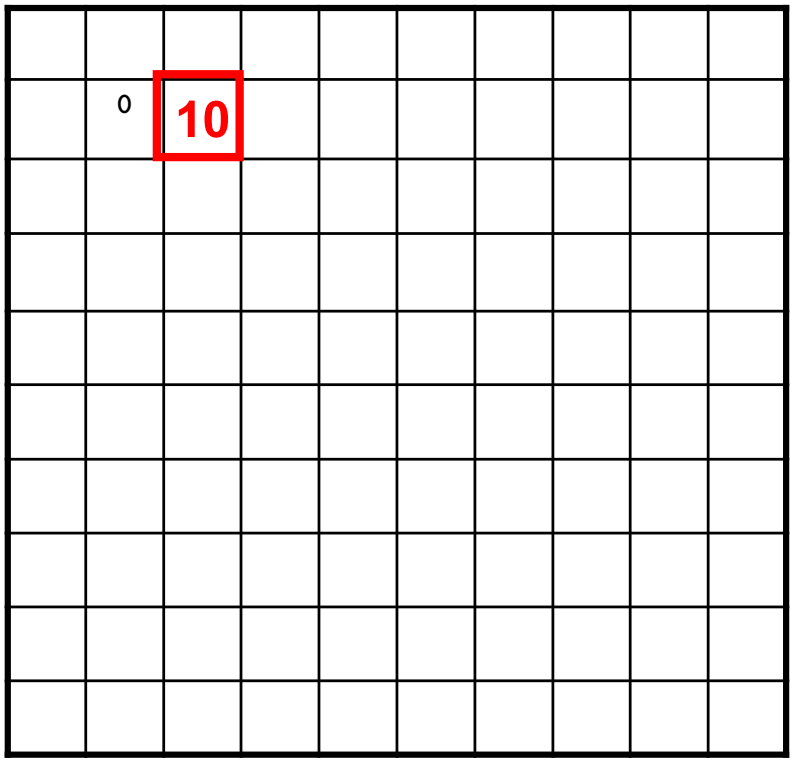
Convolution Operation



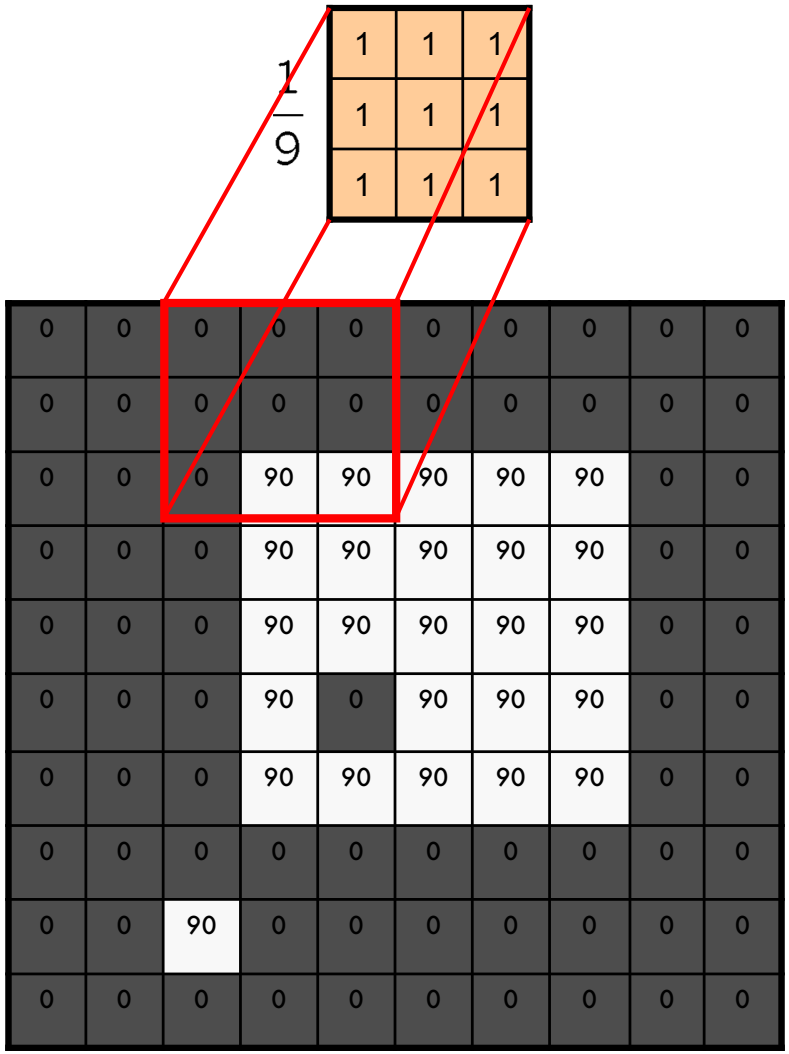


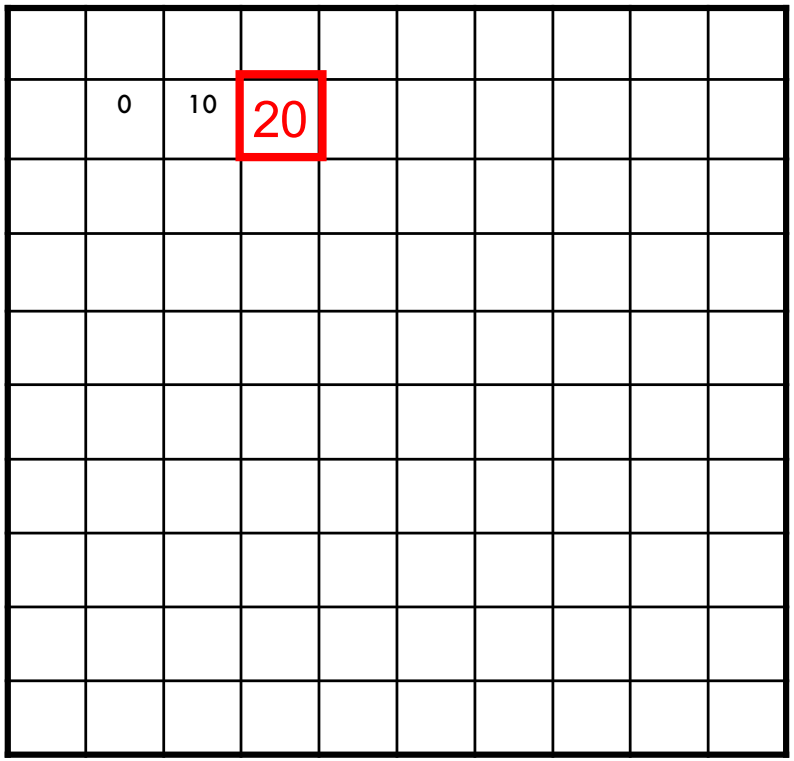
Convolution Operation



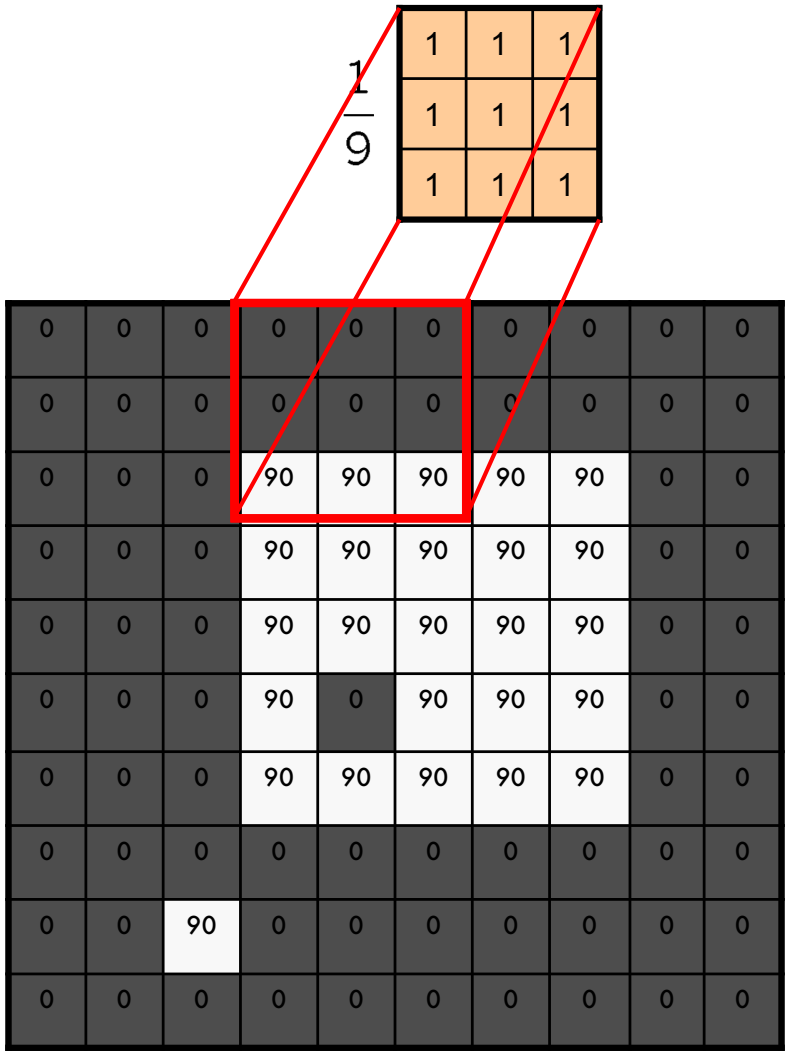


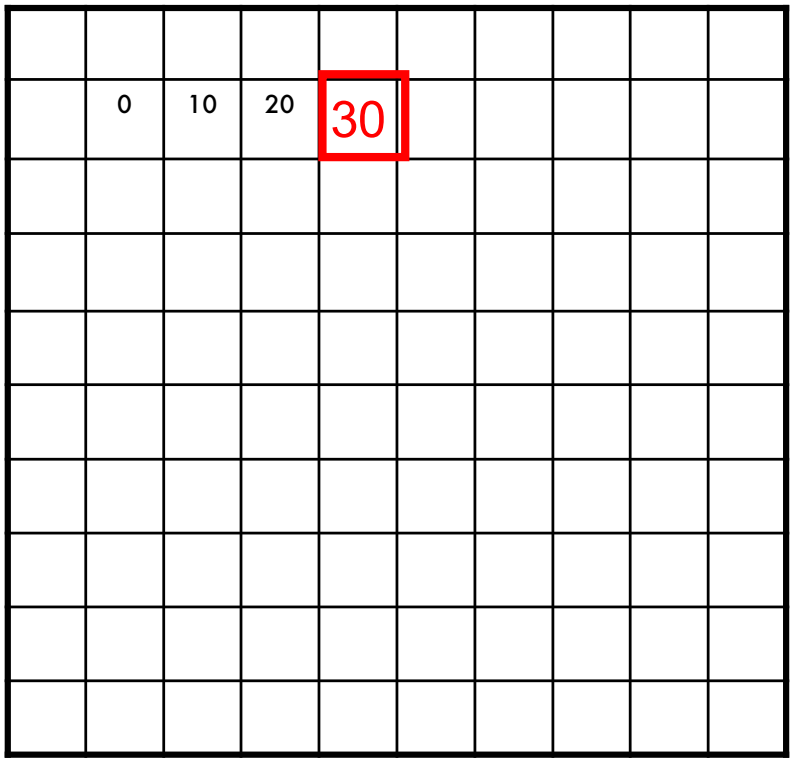
Convolution Operation



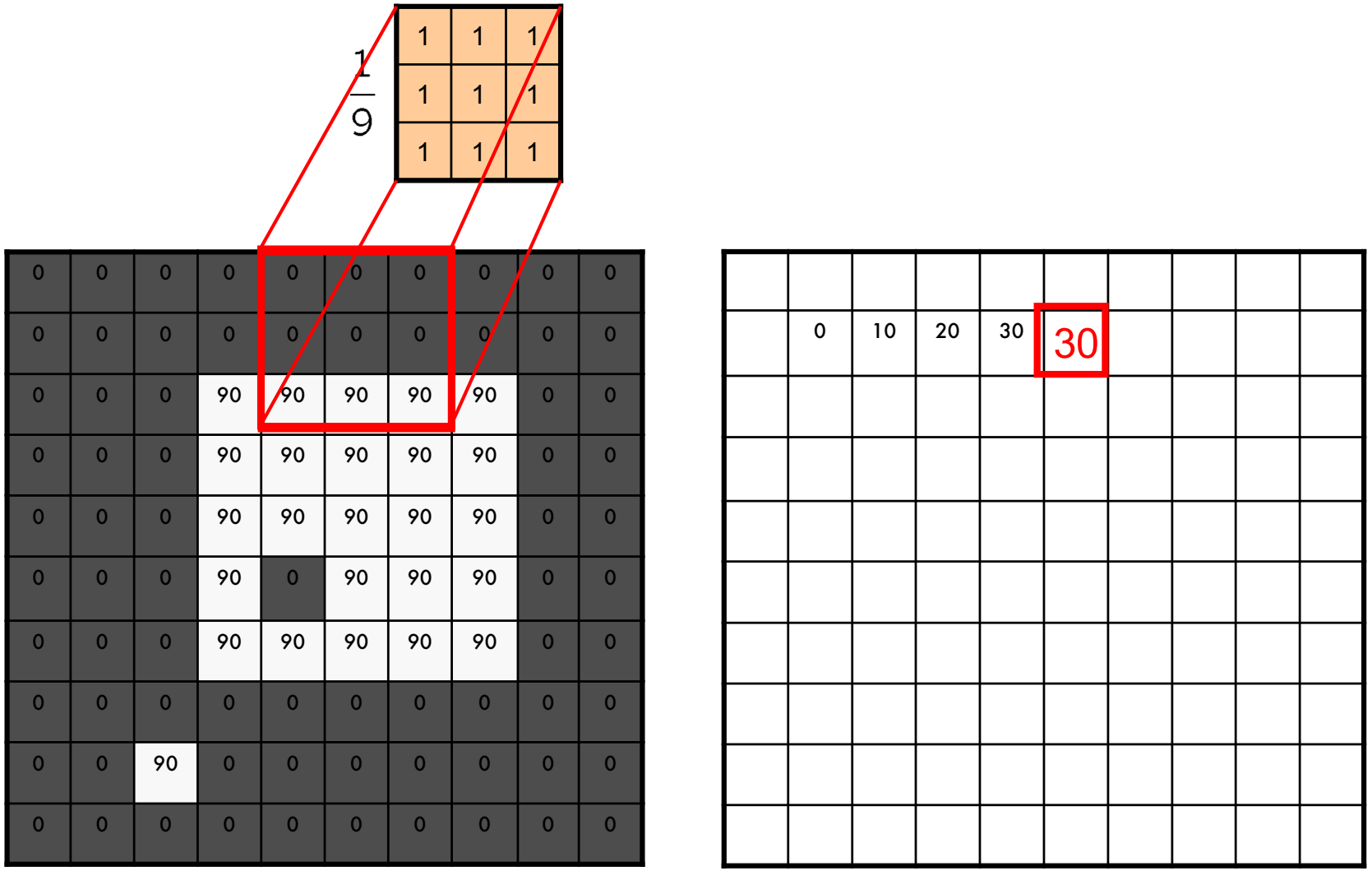


Convolution Operation

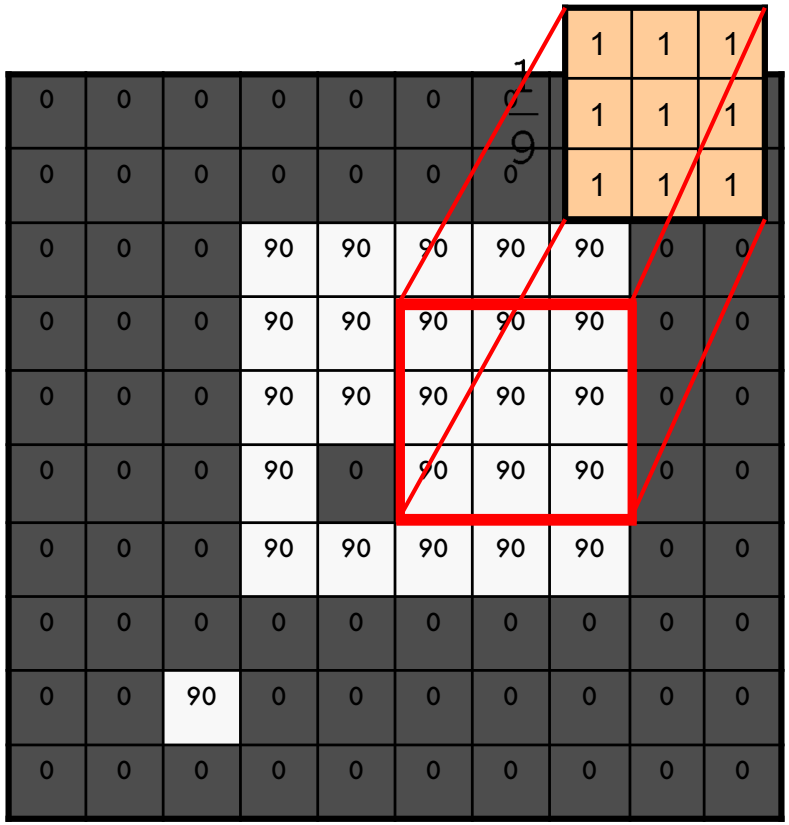


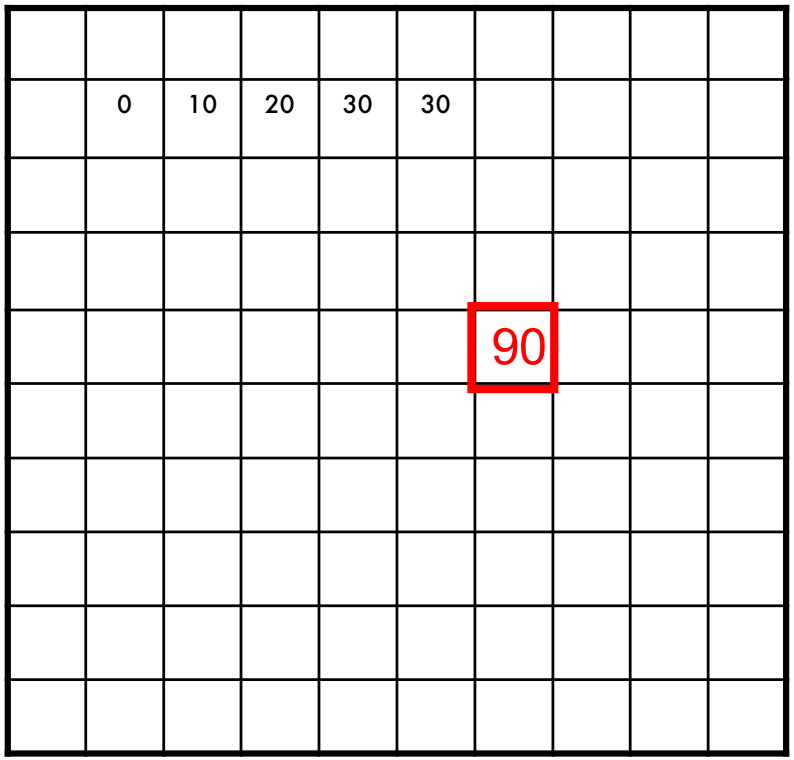


Convolution Operation



Convolution Operation





Convolution Operation

Filter

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

2D Image

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

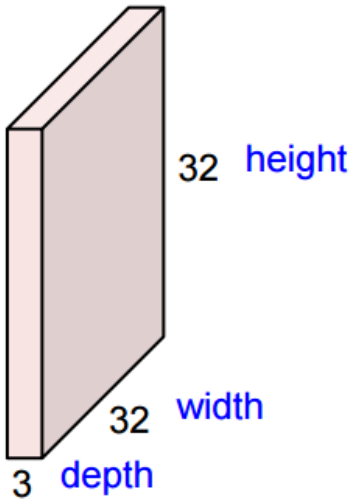
Obtained activation map after applying the filter

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

Convolutional Layer with RGB Images

RGB image (3D)

32x32x3 image



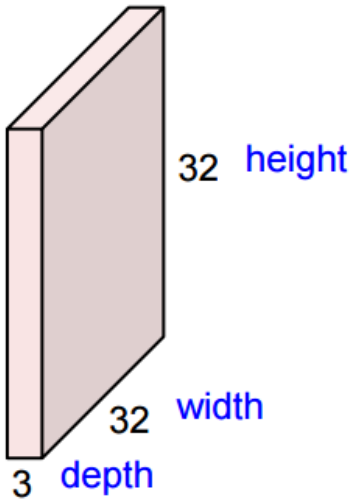
5x5x3 filter

Filters size are always odd!

Convolutional Layer with RGB Images

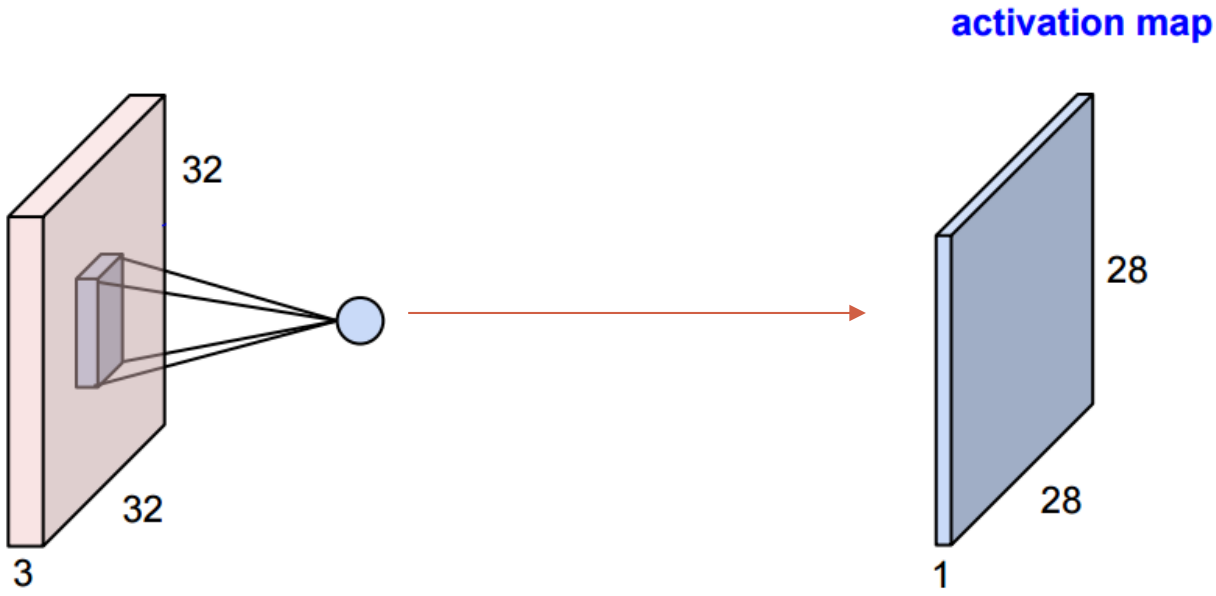
RGB image (3D)

32x32x3 image



5x5x3 filter

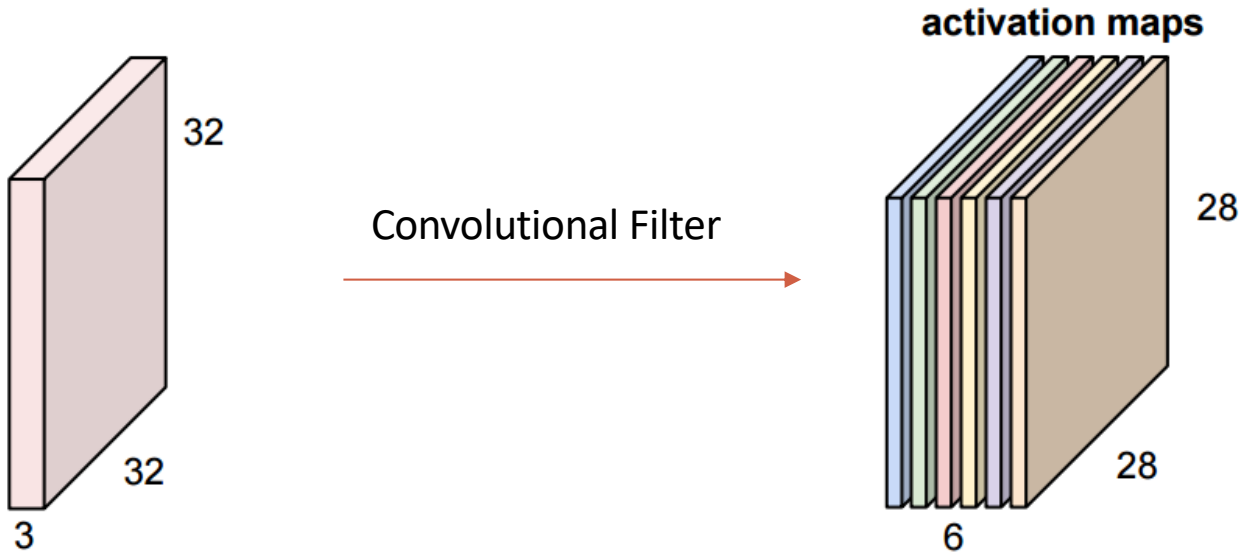
Convolutional Layer with RGB Images



Sum over 5x5x3 = 75 multiplications

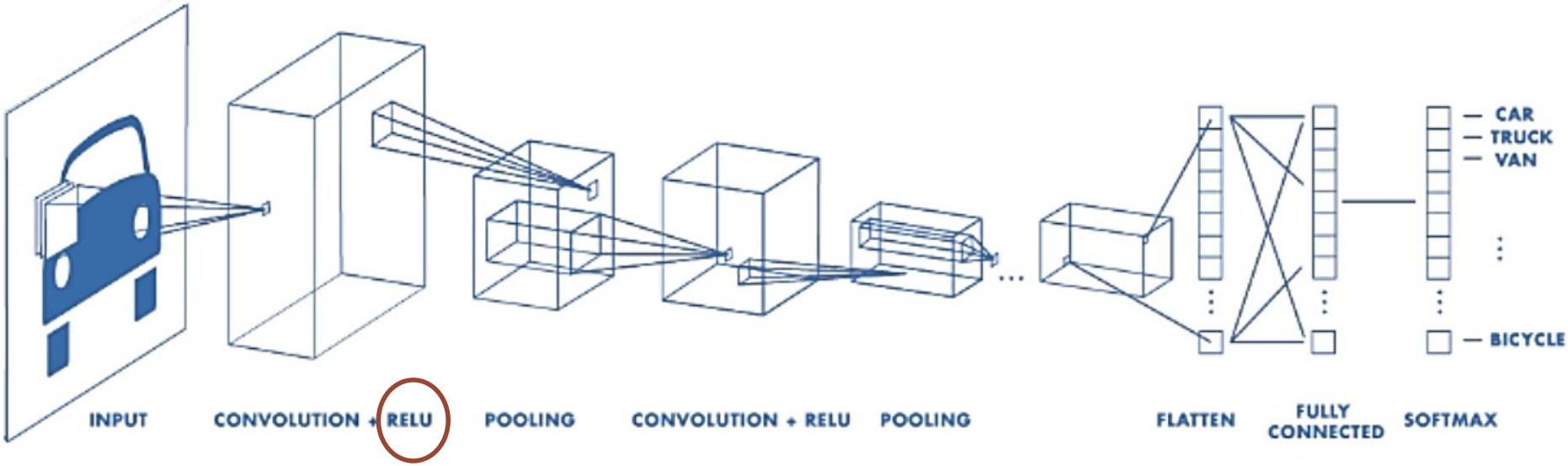
Convolutional Layer with RGB Images

Assuming we have **six** 5x5x3 filters:



Number of Activation Maps: 6

Typical CNN Architecture



CNN Architecture: Activation Layer

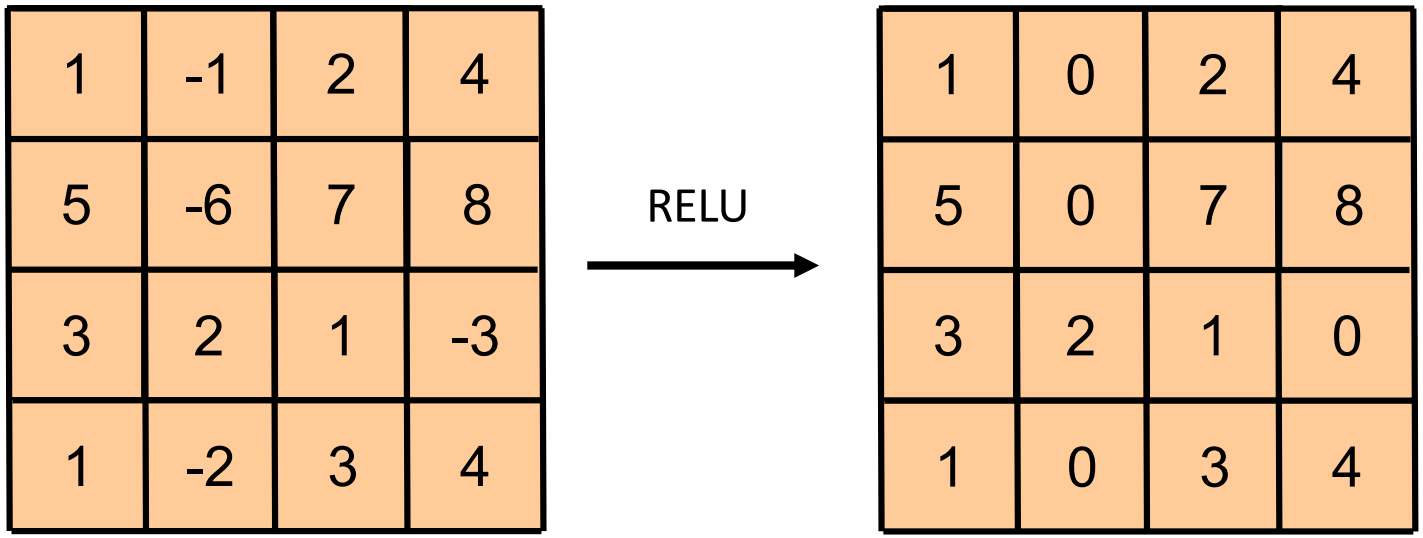
- **Intuition:** Increase the nonlinearity of the entire architecture

- A layer of neurons that applies the non-linear activation function, such as,

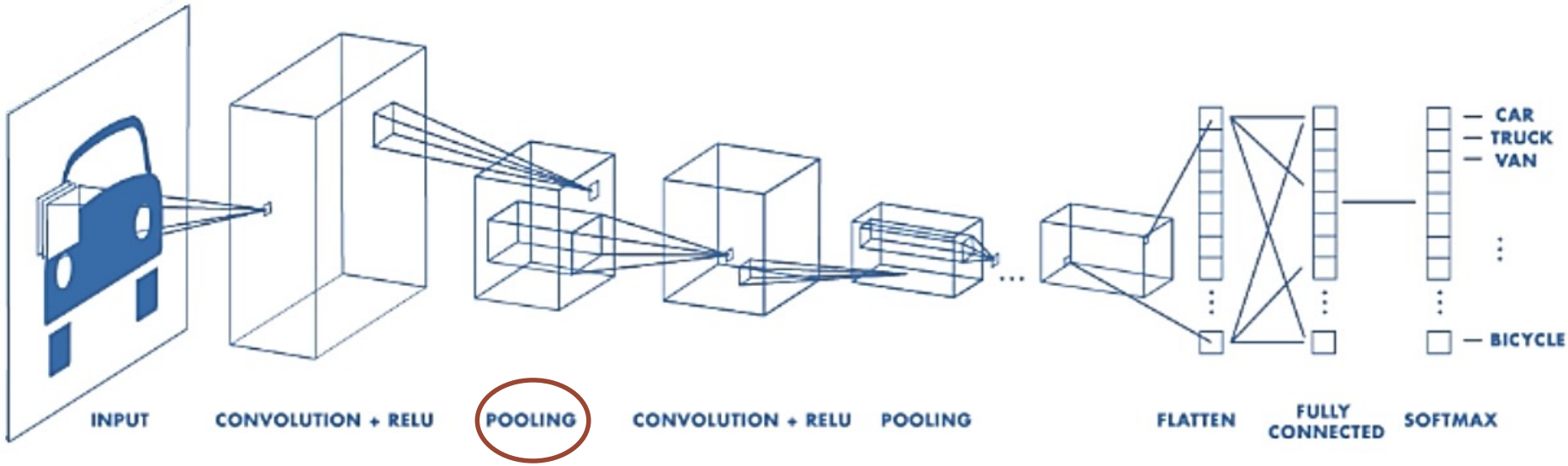
  - $f(x) = \max(0, x)$ = **The most common: Linear rectifier Unit (RELU)**
  - $f(x) = \tanh x$ (hyperbolic tangent)
  - $f(x) = |\tanh x|$
  - $f(x) = (1 + e^{-x})^{-1}$ (sigmoid function)

Example for RELU operation

$f(x) = \max(0, x)$ = **The most common: Linear rectifier Unit (RELU)**

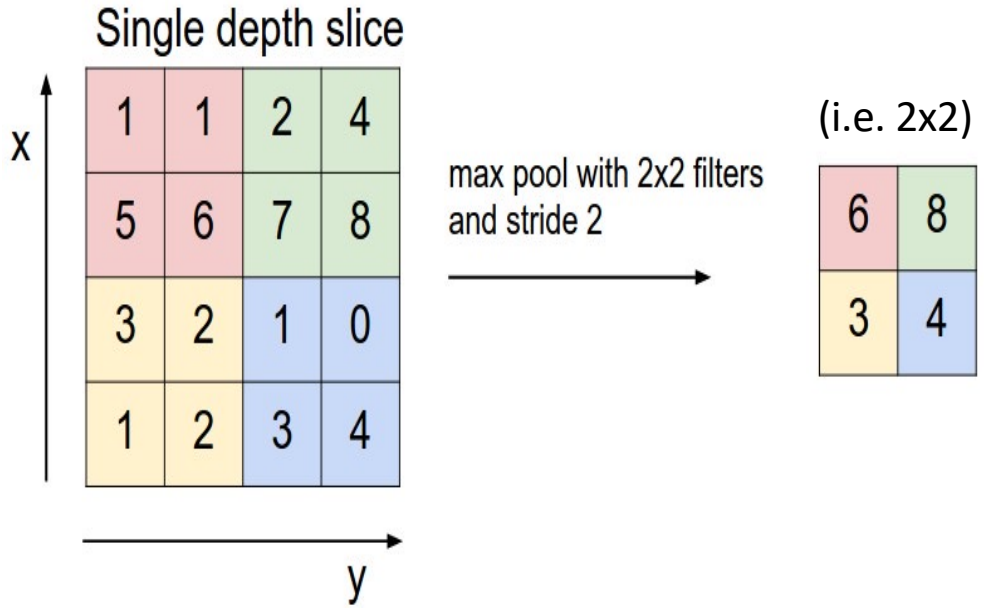


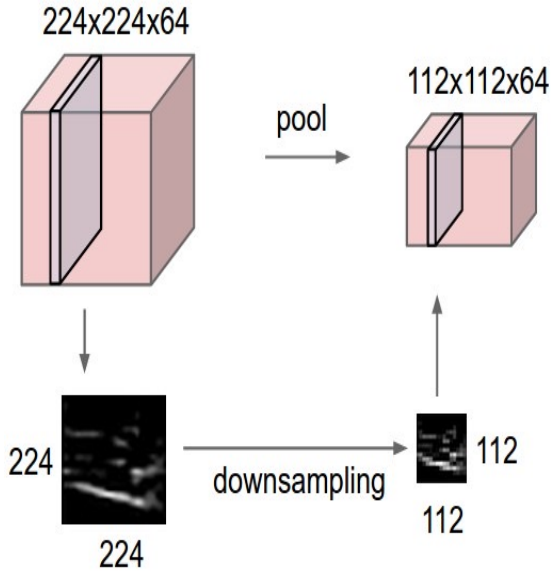
Typical CNN Architecture



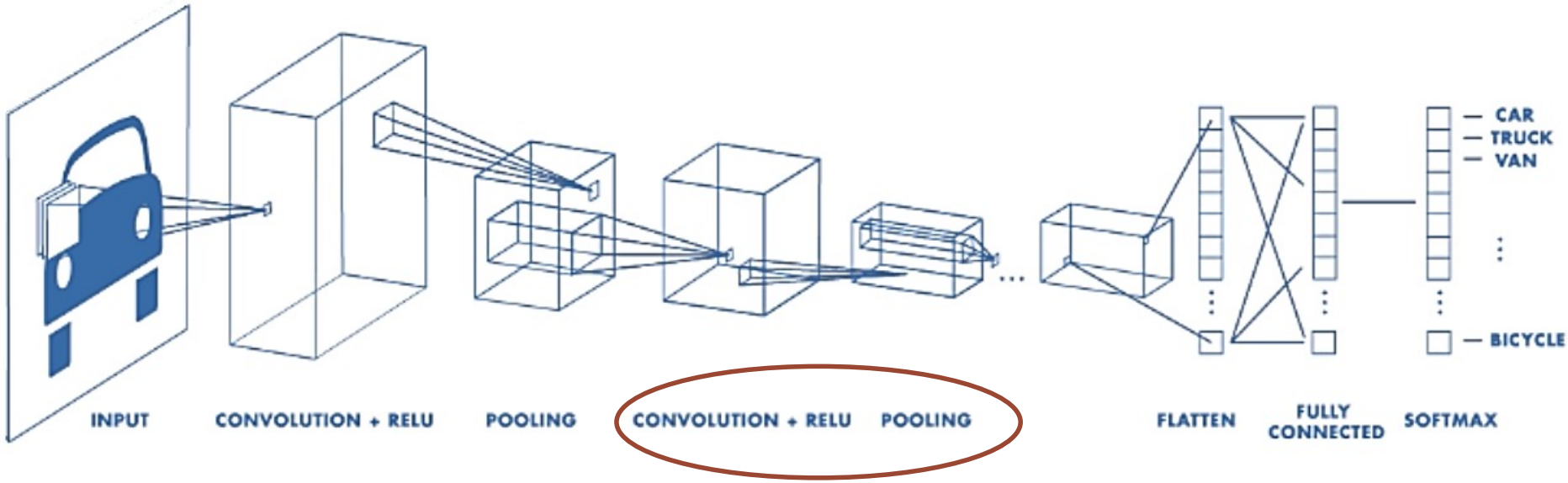
CNN Architecture: Pooling Layer

- **Intuition:** to progressively reduce the spatial size of the representation to **reduce the amount of parameters and computation time in the network**.
- Max-Pooling, Min-Pooling, Average-Pooling, etc.
- **Max-Pooling** partitions the input image into a set of non-overlapping rectangles and, for each such sub-region, outputs the **maximum value (for max pooling)**
- **Filter size and stride information should be given.**



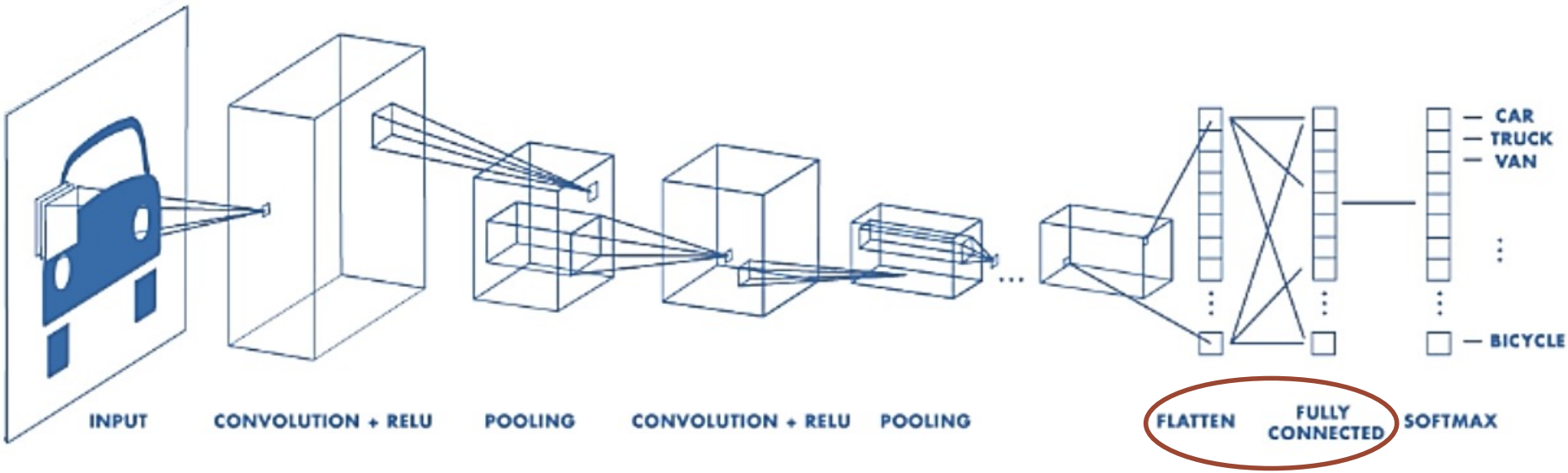


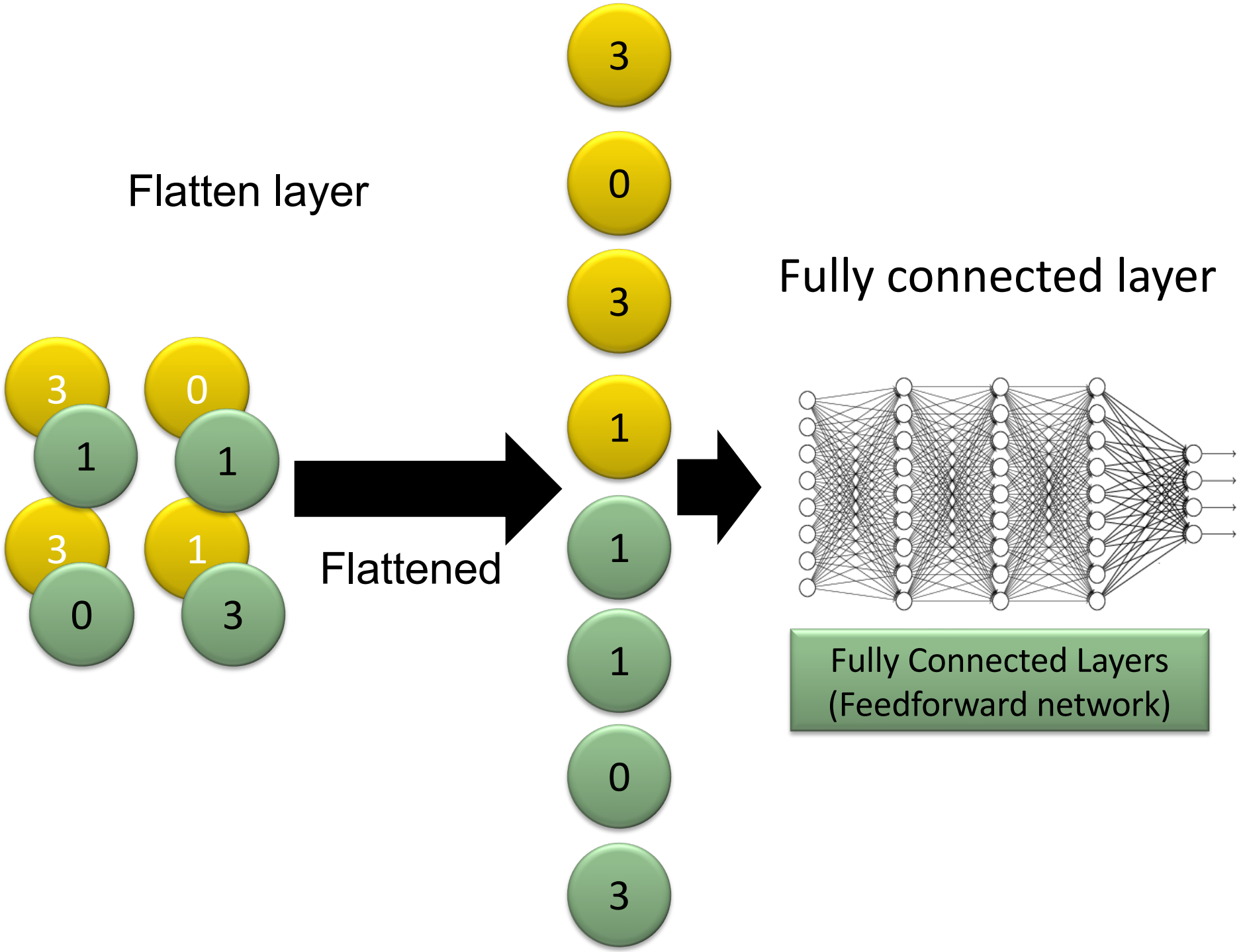
Typical CNN Architecture



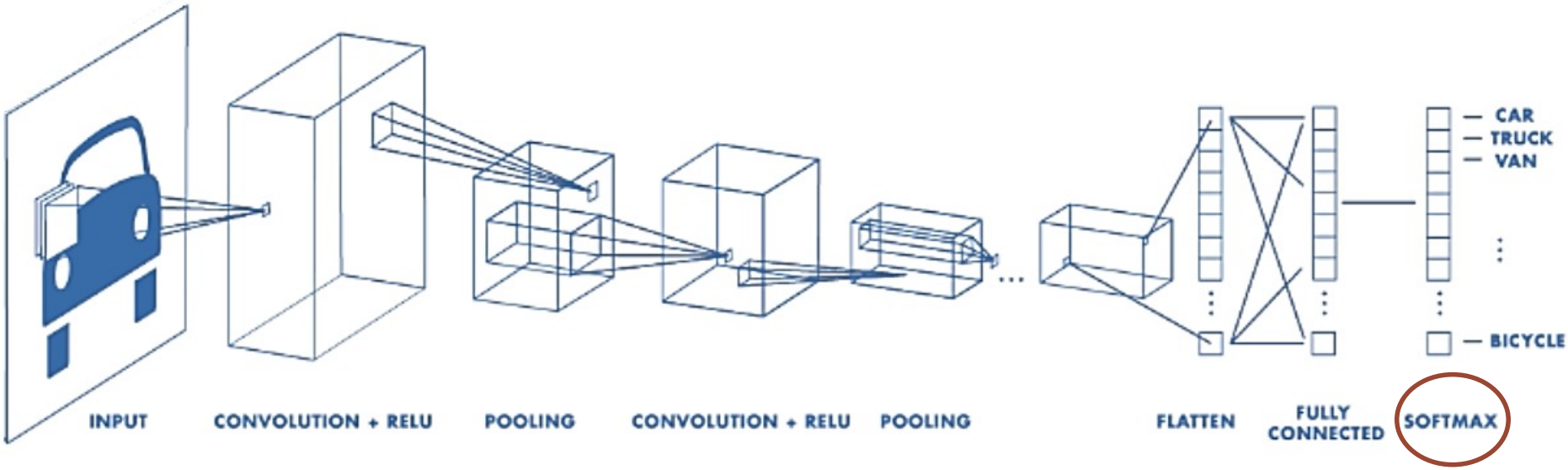
May repeat many times

Typical CNN Architecture




Flatten layer
3
0
1
1
3
1
0
3
Flattened
3
0
3
1
1
1
0
3
Fully connected layer
Fully Connected Layers
(Feedforward network)

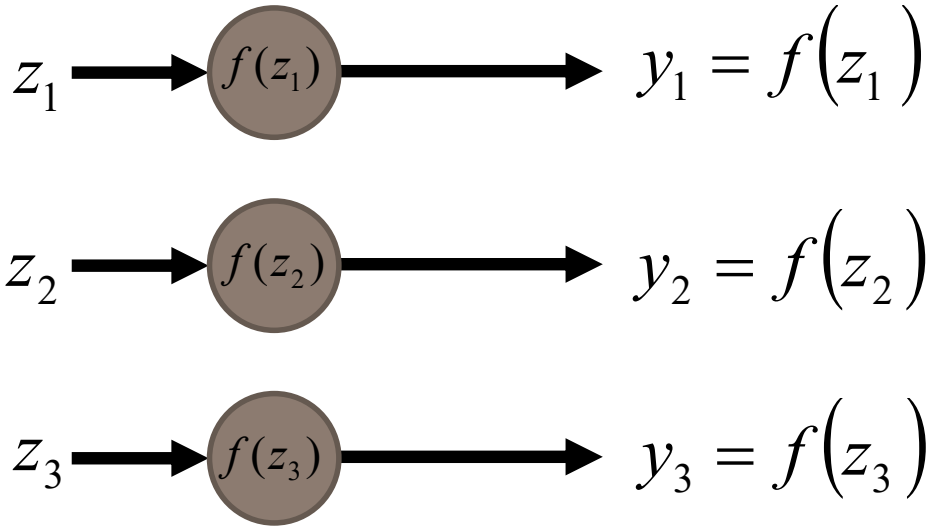
Typical CNN Architecture



Softmax Layer

- Softmax layer as the output layer

Ordinary Layer



In general, the output of network can be any value.

May not be easy to interpret

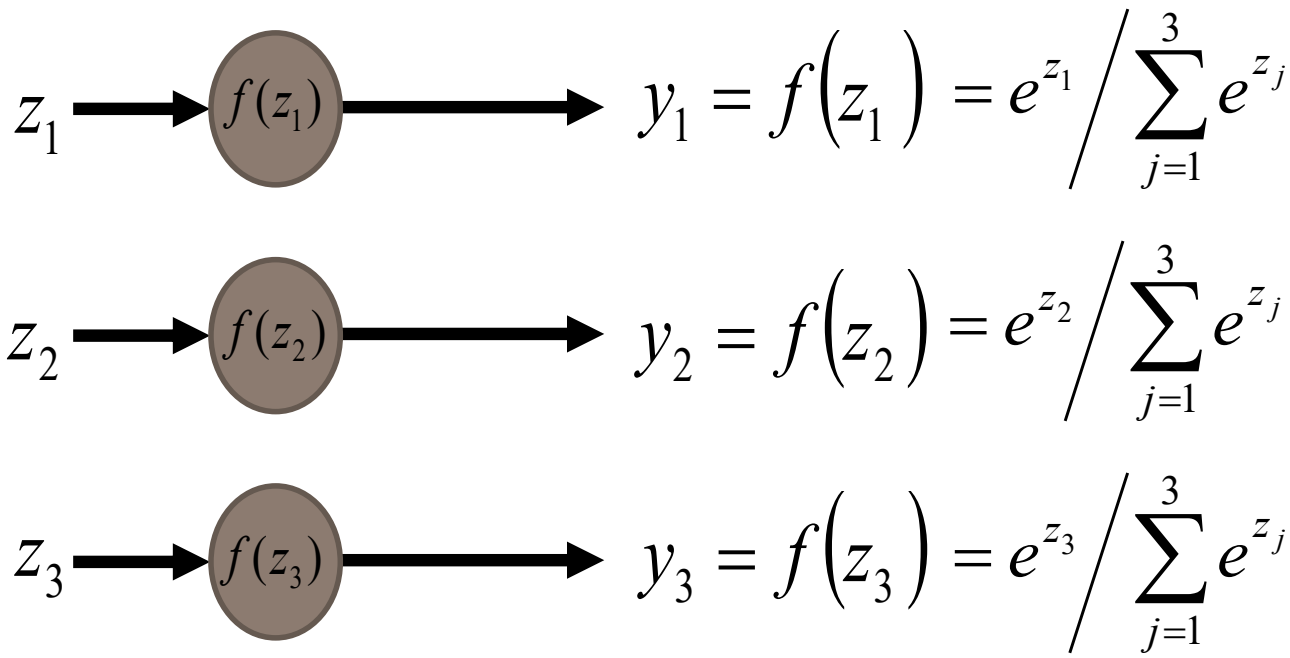
Softmax Layer

- Softmax layer as the output layer

Probability:

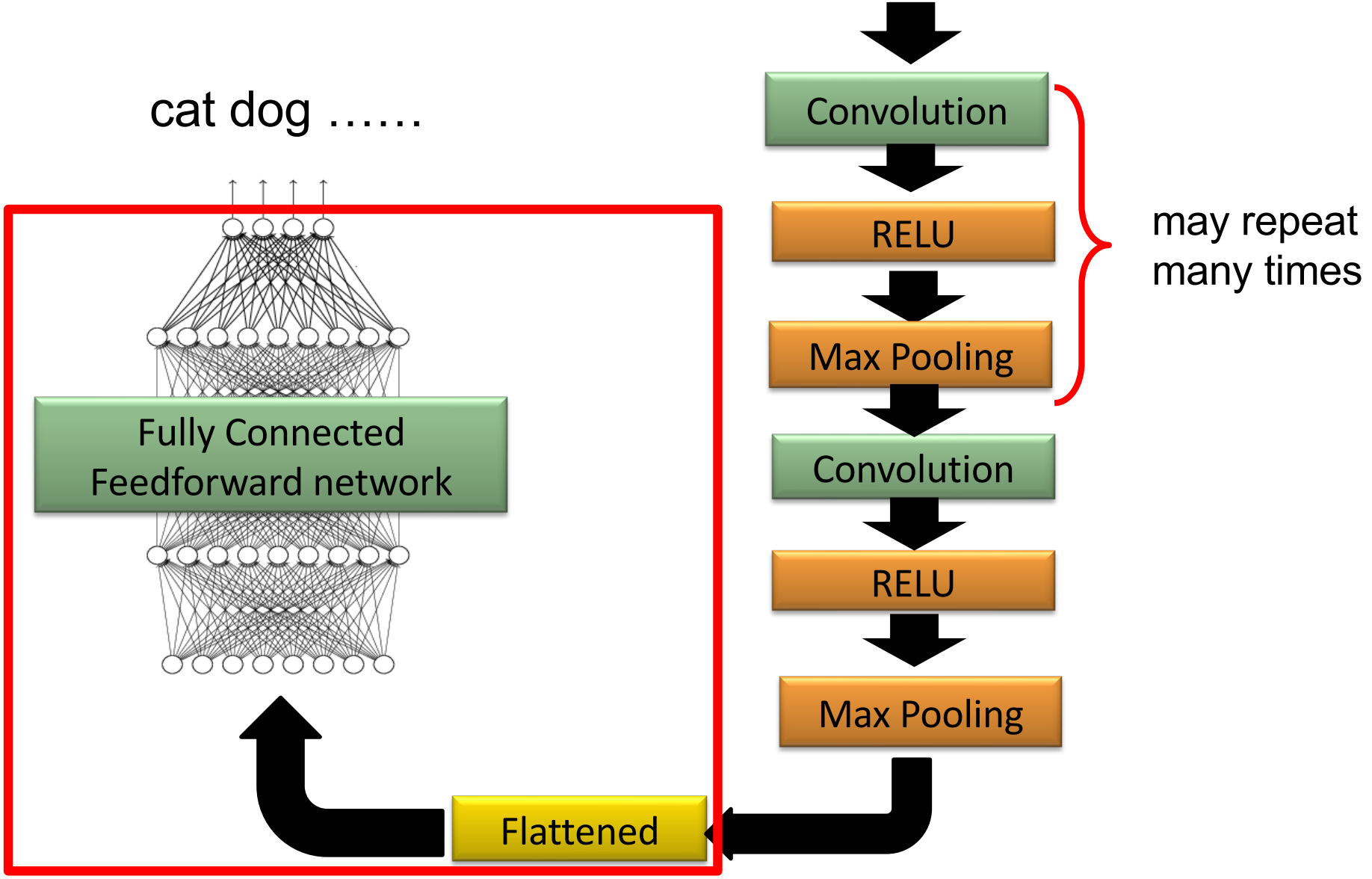
- $1 > y_i > 0$
- $\sum_i y_i = 1$

Softmax Layer



Example: $y_1 = 0.87,\ y_2 = 0.12,\ y_3 = 0.01$

The whole CNN



The whole CNN

Convolution

RELU

Max Pooling

new images

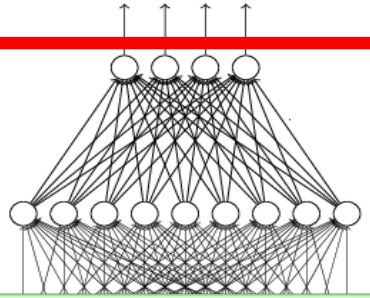
Convolution

RELU

Max Pooling

new images

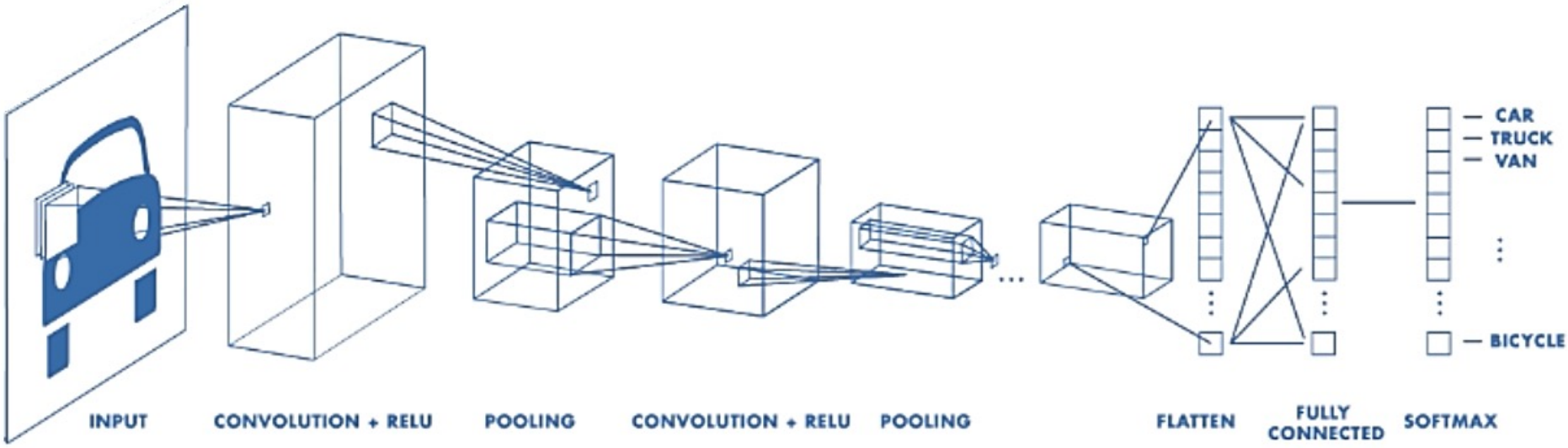
cat dog ……



Fully Connected Feedforward network

Flattened

Typical CNN Architecture



CNN Training

- CNN is trained the same way like traditional Neural Networks: **backpropagation** with **gradient descent**. Due to the convolution operation it's more mathematically involved.

$$E(\mathbf{w}) = \sum_{j=1}^{N} \left(y_j - \mathrm{t}_j \right)^2 \qquad \mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial E}{\partial \mathbf{w}}$$

- **Back-propagation:** gradients are computed in the direction from output to input layers and combined using chain rule

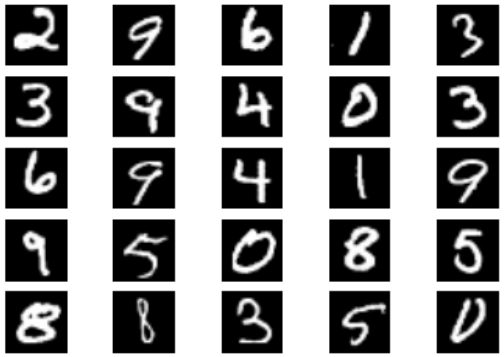
- **Generally Stochastic gradient descent is used:** compute the weight update w.r.t. one training example (or a small **batch** of examples) at a time, cycle through **randomly selected** training examples in **multiple iterations (epochs)**.

CNN Architecture

- **Intuition:** Neural network with specialized connectivity structure,
  - Stacking multiple layers of feature extractors
  - Low-level layers extract local features.
  - High-level layers extract global patterns.
- A CNN is a list of layers that transform the input data into an output class/prediction.

Image Classification via CNN

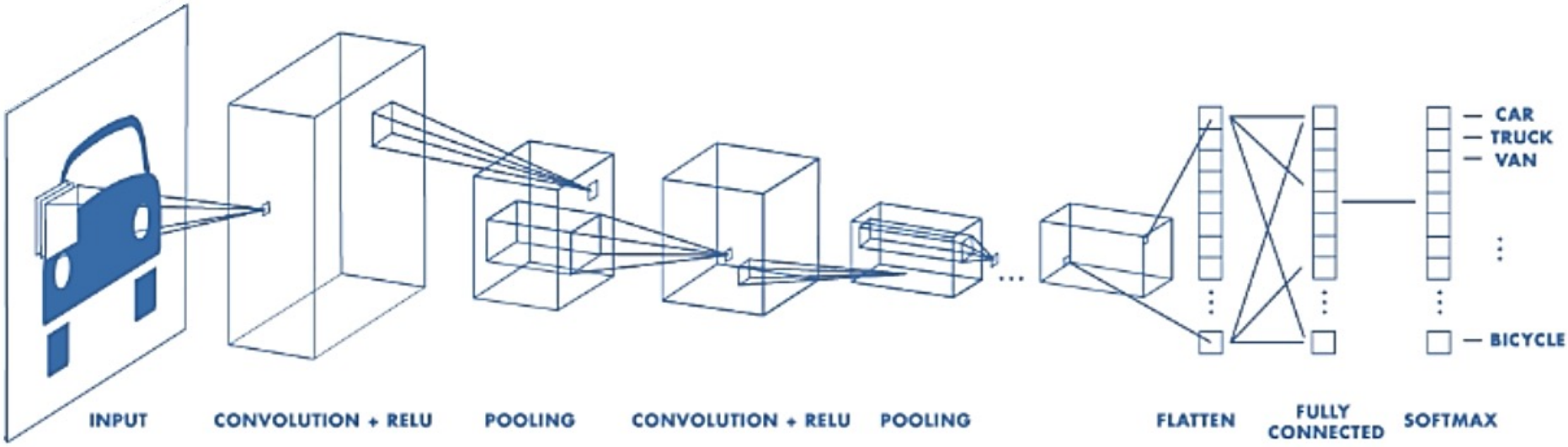
- K classes
- Task: assign correct class label to the whole image



Digit classification (MNIST)

Object recognition (Caltech-101)

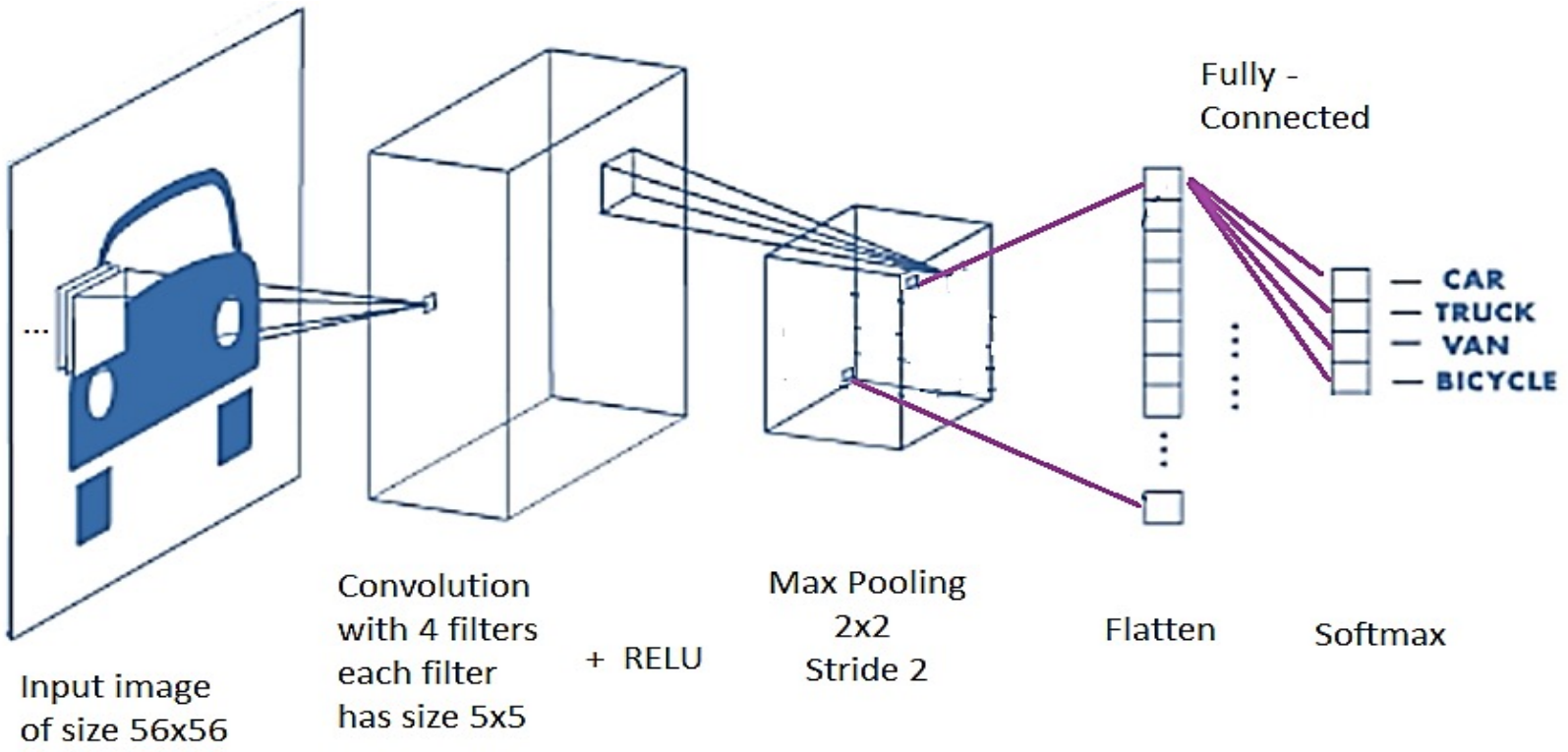
Typical CNN Architecture for Classification



Learnable Parameter Calculation in CNN

Example 1

Answer the following questions according to the CNN model given below:



(Each convolution output has size 52x52 since the filter size is 5x5)

Questions

a) How many weights in the convolutional layer do we need to learn?

b) How many activation maps (convolution outputs) are produced?

c) How many RELU operations are performed?

d) What will be the size of single slice after the Max-Pooling operation?

e) What will be the number of neurons (units) after the flatten operation?

f) The Softmax layer has 4 units since there are 4 classes in this CNN. We have fully connection from flatten units to Softmax units. How many connections present in total between these two layers?

g) Why do we need Softmax layer at the classification stage? Explain.

Solution

a) How many weights in the convolutional layer do we need to learn?

4 filters $\times 5 \times 5 = 100$ weights

b) How many activation maps (convolution outputs) are produced?

We have 4 filters so 4 outputs (activation maps) will be produced after convolution layer.

c) How many RELU operations are performed?

After convolution layer, each activation map has size 52×52. There are 4 activation maps. We apply RELU to all pixels: $52 \times 52 \times 4 = 10816$ RELU operations.

d) What will be the size of single slice after the Max-Pooling operation?

Since the block size is 2×2 and sride is 2 $(\text{i.e. } 2 \times 2)$ the size of each slice will be 26×26 $(half\ of\ 52 \times 52)$.

e) What will be the number of neurons (units) after the flatten operation?

We have 4 slices after Max-Pooling (each has size 26×26). After flattening, there will be $26 \times 26 \times 4 = 2704\ units$ on flaten layer.

f) The Softmax layer has 4 units since there are 4 classes in this CNN. We have fully connection from flatten units to Softmax units. How many connections present in total between these two layers?

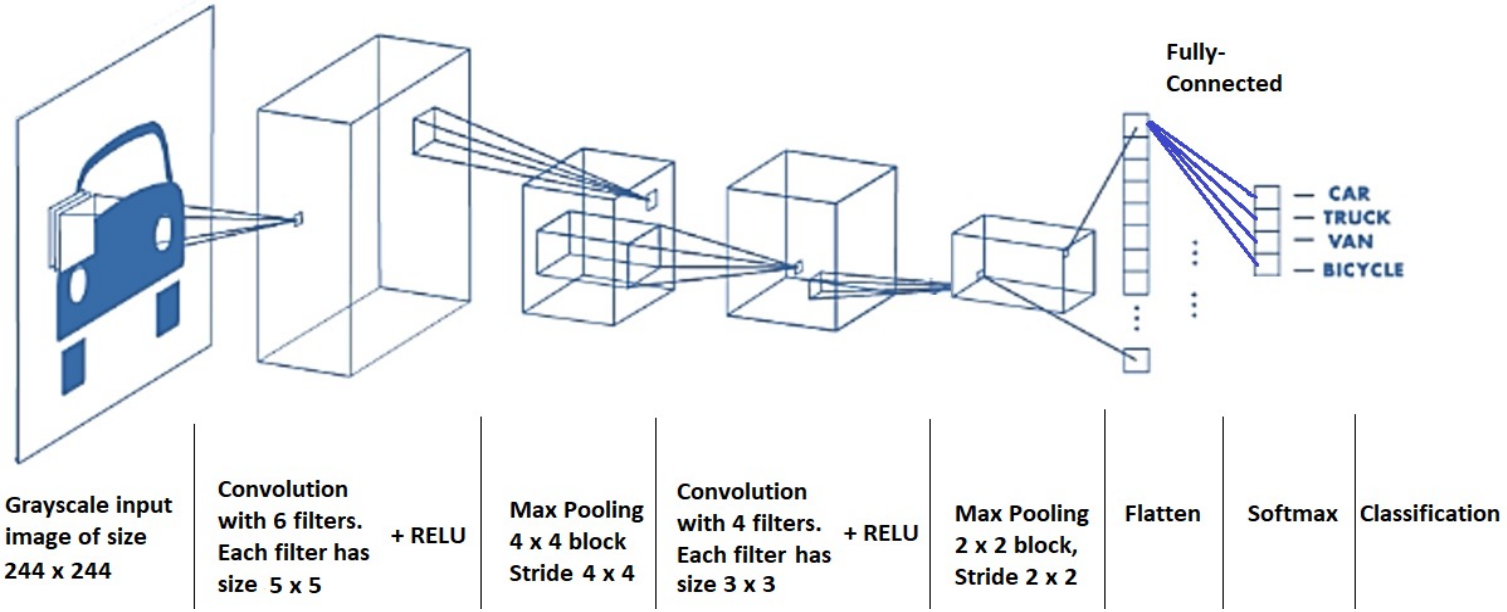
We have full connection from flaten to softmax layer that means each unit in flaten has connection to each unit in softmax layer. In total, $2704 \times 4 = 10816$ connections.

g) Why do we need Softmax layer at the classification stage? Explain.

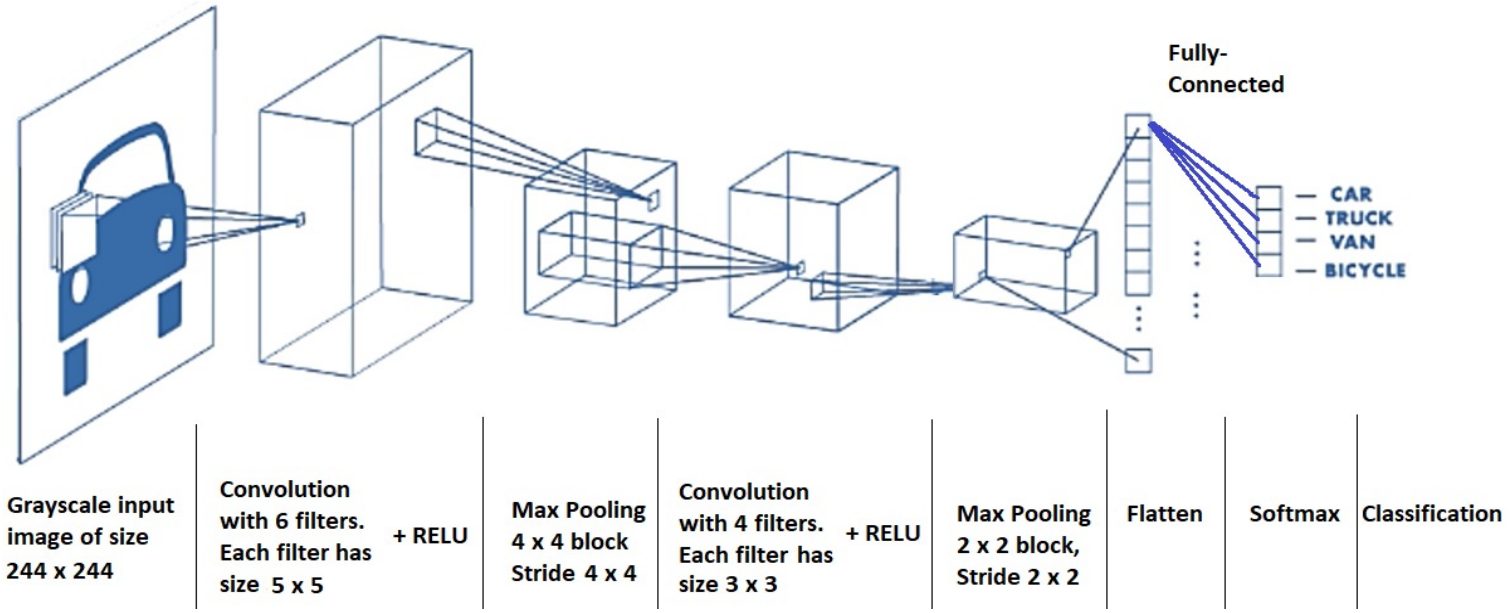
We need softmax layer to convert output numbers to probability values between 0 and 1 for each class. The propability values indicate confidence scores for each class.

Example 2

A diagram of a Convolutional Neural Network (CNN) is given below. The CNN converts an image of size 244x244 into 4 output values (i.e. 4 classes). The network has the layers and operations from input to output as shown below in the figure. Please read the parameters of these operations carefully, and answer the following questions about the network.



Note: During the filtering process in convolution layers, the filter does not move outside the input image. For example, in the first convolution layer, each convolution output image has size 240x240 since the filter size is 5x5.



a) How many weights in the convolutional layers do we need to learn?

b) How many activation maps (convolution outputs) are produced after the first convolutional layers, and after the second convolutional layers?

c) What will be the size of single slice after the first Max-Pooling operation?

d) What will be the size of single slice after the second Max-Pooling operation?

e) In total, how many RELU operations are performed in the convolutional layers?

f) What will be the number of neurons (units) after the flatten operation?

g) Why do we need Softmax layer at the classification stage? Shortly explain.

h) The Softmax layer has 4 units since there are 4 classes in this CNN. We have fully connection from flatten units to Softmax units. How many connections present in total between these two layers?

Example 3

- 2D input, I:

```
I = [[1, 2, 0, 1],
     [3, 1, 2, 2],
     [0, 1, 3, 1],
     [2, 2, 1, 0]]
```

- Convolution kernel (filter):

```
K = [[1,  0, -1],
     [0, -1,  0],
     [-1, 0,  1]]
```

Stride: 1

Padding: 0 (valid convolution)

Activation: ReLU

Pooling: 2×2 Max Pooling, stride 2

- **Output[0][0] = Top-left 3×3 patch:**

```lua
Patch:
[[1, 2, 0],
 [3, 1, 2],
 [0, 1, 3]]
```

Element-wise multiply with kernel `K`:

```markdown
(1×1) + (2×0) + (0×-1) +
(3×0) + (1×-1) + (2×0) +
(0×-1) + (1×0) + (3×1)

= 1 + 0 + 0 + 0 -1 + 0 + 0 + 0 + 3 = **3**
```

Output[0][1]:

Patch:

```lua
[[2, 0, 1],
 [1, 2, 2],
 [1, 3, 1]]
```

Calculation:

```markdown
2×1 + 0×0 + 1×(-1) +
1×0 + 2×(-1) + 2×0 +
1×(-1) + 3×0 + 1×1
= 2 + 0 + (-1) + 0 + (-2) + 0 + (-1) + 0 + 1
= 2 -1 -2 -1 + 1 = **-1**
```

Output[1][0]:

Patch:

```lua
[[3, 1, 2],
 [0, 1, 3],
 [2, 2, 1]]
```

Calculation:

```markdown
3×1 + 1×0 + 2×(-1) +
0×0 + 1×(-1) + 3×0 +
2×(-1) + 2×0 + 1×1
= 3 + 0 + (-2) + 0 + (-1) + 0 + (-2) + 0 + 1
= 3 -2 -1 -2 + 1 = **-1**
```

◆ Output[1][1]:

Patch:

```lua
[[1, 2, 2],
 [1, 3, 1],
 [2, 1, 0]]
```

Calculation:

```markdown
1×1 + 2×0 + 2×(-1) +
1×0 + 3×(-1) + 1×0 +
2×(-1) + 1×0 + 0×1
= 1 + 0 + (-2) + 0 + (-3) + 0 + (-2) + 0 + 0
= 1 -2 -3 -2 = **-6**
```

📥 Convolution Output Before Activation:

```lua
[[ 3, -1],
 [-1, -6]]
```

⚡ Step 2: ReLU Activation

ReLU(x) = max(0, x)

```lua
[[3,  0],
 [0,  0]]
```

🔁 Step 3: Max Pooling (2×2, stride 2)

The feature map is already 2×2, so applying 2×2 max pooling

```lua
Max of [[3, 0],
        [0, 0]] = **3**
```

✅ Final Output:

After convolution → ReLU → max pooling:

```lua
Output = [[3]]
```